

目次

本書について	6
改正履歴一覧表	6
改訂履歴	7
用語の説明	8
1. 試験の概要	9
1.1 目的	9
1.2 試験対象	10
1.3 試験環境	11
1.4 試験の種類	13
2. API 仕様	14
2.1 API の概要	14
2.2 各 API の仕様	15
2.2.1 GET /health (稼働状態を確認する)	15
2.2.2 GET /books (蔵書を検索する)	15
2.2.3 POST /books (蔵書を登録する)	15
2.2.4 GET /books/{bookId} (蔵書を 1 件取得する)	16
2.2.5 GET /members/{memberId} (会員と貸出状況を取得する)	16
2.2.6 GET /members/{memberId}/loans (会員の貸出履歴を取得する)	16
2.2.7 POST /loans (本を貸し出す)	17
2.2.8 POST /loans/{loanId}/return (本を返却する)	17
2.3 データ定義	19
2.3.1 Health	19
2.3.2 Book	19
2.3.3 BookCreate	19
2.3.4 BookList	19
2.3.5 Member	20
2.3.6 LoanCreate	20
2.3.7 Loan	20
2.3.8 LoanList	20
2.3.9 Error	20
3. 試験の設計	21
3.1 資産の流れ	21
3.2 初期データ	22
3.3 PL/SQL 前後処理	23
3.3.1 実行順序	23
3.3.2 記述上の制約	23
3.3.3 使用している PL/SQL 構文	24
3.4 モックの返し分け	25

4. 実施手順	26
4.1 準備	26
4.2 実行	27
4.3 証跡	28
4.4 終了	29
5. 契約テスト	30
5.1 前処理・後処理（全 suite 共通）	30
5.2 契約 getBook 蔵書を 1 件取得する（モック・実 API の両方で実行する）	31
5.2.1 テストシナリオ	31
5.2.2 ケースデータ	31
5.2.3 HTTP 呼び出し	31
5.2.4 期待値・検証条件	32
5.3 契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する）	33
5.3.1 テストシナリオ	33
5.3.2 ケースデータ	33
5.3.3 HTTP 呼び出し	33
5.3.4 期待値・検証条件	33
5.4 契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する）	35
5.4.1 テストシナリオ	35
5.4.2 ケースデータ	35
5.4.3 HTTP 呼び出し	35
5.4.4 期待値・検証条件	36
5.5 契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する）	38
5.5.1 テストシナリオ	38
5.5.2 ケースデータ	38
5.5.3 HTTP 呼び出し	39
5.5.4 期待値・検証条件	39
5.6 契約 listMemberLoans 会員の貸出履歴を取得する（モック・実 API の両方で実行する）	41
5.6.1 テストシナリオ	41
5.6.2 ケースデータ	41
5.6.3 HTTP 呼び出し	41
5.6.4 期待値・検証条件	42
5.7 契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）	43
5.7.1 テストシナリオ	43
5.7.2 ケースデータ	43
5.7.3 HTTP 呼び出し	43
5.7.4 リクエストボディ	43
5.7.5 期待値・検証条件	44
5.8 契約 createLoan 本を貸し出す（モック・実 API の両方で実行する）	46
5.8.1 テストシナリオ	46

5.8.2 ケースデータ	46
5.8.3 HTTP 呼び出し	46
5.8.4 リクエストボディ	47
5.8.5 期待値・検証条件	47
5.9 契約 returnLoan 本を返却する（モック・実 API の両方で実行する）	49
5.9.1 テストシナリオ	49
5.9.2 ケースデータ	49
5.9.3 HTTP 呼び出し	49
5.9.4 リクエストボディ	50
5.9.5 期待値・検証条件	50
6. シナリオ試験	52
6.1 LIB-001 貸出～返却の正常系	52
6.1.1 テストシナリオ	53
6.1.2 前処理	54
6.1.3 後処理	54
6.1.4 HTTP 呼び出し	54
6.1.5 リクエストボディ	55
6.1.6 期待値・検証条件	55
6.2 LIB-002 在庫切れ	59
6.2.1 テストシナリオ	59
6.2.2 前処理	60
6.2.3 後処理	60
6.2.4 HTTP 呼び出し	60
6.2.5 リクエストボディ	61
6.2.6 期待値・検証条件	61
6.3 LIB-003 延滞中の貸出拒否	64
6.3.1 テストシナリオ	64
6.3.2 前処理	65
6.3.3 後処理	65
6.3.4 HTTP 呼び出し	65
6.3.5 リクエストボディ	66
6.3.6 期待値・検証条件	66
6.4 LIB-004 貸出上限	69
6.4.1 テストシナリオ	69
6.4.2 前処理	70
6.4.3 後処理	70
6.4.4 HTTP 呼び出し	70
6.4.5 リクエストボディ	70
6.4.6 期待値・検証条件	71
6.5 LIB-005 SAVEPOINT による部分確定	73

6.5.1	テストシナリオ	73
6.5.2	前処理	75
6.5.3	後処理	75
6.5.4	HTTP 呼び出し	75
6.5.5	リクエストボディ	76
6.5.6	期待値・検証条件	77
6.6	LIB-006 異常系	80
6.6.1	テストシナリオ	80
6.6.2	前処理	81
6.6.3	後処理	81
6.6.4	HTTP 呼び出し	81
6.6.5	リクエストボディ	82
6.6.6	期待値・検証条件	84
6.7	LIB-007 大量の貸出履歴	87
6.7.1	テストシナリオ	87
6.7.2	前処理	87
6.7.3	後処理	88
6.7.4	HTTP 呼び出し	88
6.7.5	リクエストボディ	88
6.7.6	期待値・検証条件	89
6.8	DEMO 事後検証による不整合の検知	91
6.8.1	テストシナリオ	91
6.8.2	前処理	91
6.8.3	後処理	91
6.8.4	HTTP 呼び出し	92
6.8.5	期待値・検証条件	92
7.	実施記録	93
	不具合・所見	93
8.	付録 前後処理 SQL 全文	94
8.1	sql/common/	94
8.1.1	00_reset.sql	94
8.1.2	10_seed_master.sql	95
8.1.3	90_verify_integrity.sql	96
8.2	sql/cases/	99
8.2.1	contract_setup.sql	99
8.2.2	demo_break_invariant_after.sql	99
8.2.3	lib001_assert_after.sql	100
8.2.4	lib003_overdue_before.sql	101
8.2.5	lib004_fill_loans_before.sql	102
8.2.6	lib005_assert_after.sql	103

8.2.7 lib005_savepoint_before.sql 104

8.2.8 lib006_assert_no_loans_after.sql 106

8.2.9 lib007_history_before.sql 106

本書について

本書は、サンプルの図書貸出 API に対して行う API テストの手順書である。次の 3 つのツールを組み合わせ、**OpenAPI を正本**として API 仕様・モック・テスト・手順書を 一貫して作る流れを示す。

- **runnora** … runn を組み込んだ API シナリオテスト実行基盤。runbook の前後に Oracle の SQL/PL/SQL を実行できる
- **oapi2wire** … OpenAPI と返し分け定義 (case YAML) から WireMock のモック資産を生成する
- **runnora-docgen** … runbook・ケース JSON・前後処理 SQL から、本書の手順表 (Quarto 原稿) を生成する

本書の表のうち「生成」と記したものは `scripts/build-docs.ps1` が自動生成した原稿であり、runbook や SQL を直せば再生成で追隨する。良否欄は実施者が記入する（実行結果は自動転記しない）。

改正履歴一覧表

改正番号	改正日	改正の根拠	改正の概要
基本版	2026-09-26	-	新規制定

改訂履歷

用語の説明

用語	説明
runbook	runn/runnora のシナリオ定義 (YAML)。HTTP 呼び出し・DB 照会・検証式を手順として並べる
suite / template	1 つの API を複数のケース JSON で繰り返し呼ぶ構成。suite がケースを読み、template が 1 回の呼び出しを定義する
ケース JSON	1 回の呼び出しの入力 (パス・クエリ・ボディ) と期待値 (ステータス・本文の一部)
前処理 / 後処理	runbook の前後に runnora が実行する SQL/PL/SQL。共通 (config.yaml) とケース固有 (--before-sql / --after-sql) がある
契約テスト	同じケースをモックと実装の両方に流し、どちらも OpenAPI どおりに振る舞うことを確かめるテスト
シナリオ試験	実装 (実 DB) に対して、状態の変化を複数手順で追う業務シナリオのテスト
不変条件	どの操作の後でも成り立つべき DB の条件 (例: 貸出可能冊数 = 所蔵冊数 - 貸出中件数)

1. 試験の概要

1.1 目的

図書貸出 API が OpenAPI 定義と業務ルールどおりに動作することを、モックと実装の両面から確認する。あわせて、runnora の PL/SQL 前後処理で API テストによく使う構文（FOR LOOP、SAVEPOINT、カーソル、BULK COLLECT/FORALL、例外処理など）が期待どおり動くことを確認する。

1.2 試験対象

表 1.2-1 試験対象

対象	内容	所在
API 仕様	図書貸出 API 1.0.0 (8 operation)	openapi/library-api.yaml (第 2 章)
サンプル実装	Go + go-ora。データは Oracle Database Free (23ai) の LIBAPP スキーマ	api/, db/
モック	oapi2wire が生成した WireMock 資産	mock/

業務ルールは次のとおり。シナリオ試験（第 6 章）はこれらを 1 つずつ確認する。

表 1.2-2 業務ルール

No.	業務ルール	違反時の応答	確認するシナリオ
1	貸出可能冊数が 0 の本は借りられない	409 NO_STOCK	LIB-002
2	返却期限を過ぎた未返却の貸出がある会員は借りられない	409 OVERDUE_LOAN	LIB-003
3	同時に借りられる冊数は会員ごとの上限（既定 3）まで	409 LOAN_LIMIT_EXCEEDED	LIB-004
4	利用停止 (SUSPENDED) の会員は借りられない	409 MEMBER_SUSPENDED	LIB-005, LIB-006
5	返却済みの貸出は再返却できない	409 ALREADY_RETURNED	LIB-001
6	ISBN は重複登録できない	409 DUPLICATE_BOOK	LIB-005
7	貸出期間は貸出日から 14 日	-	LIB-001

1.3 試験環境

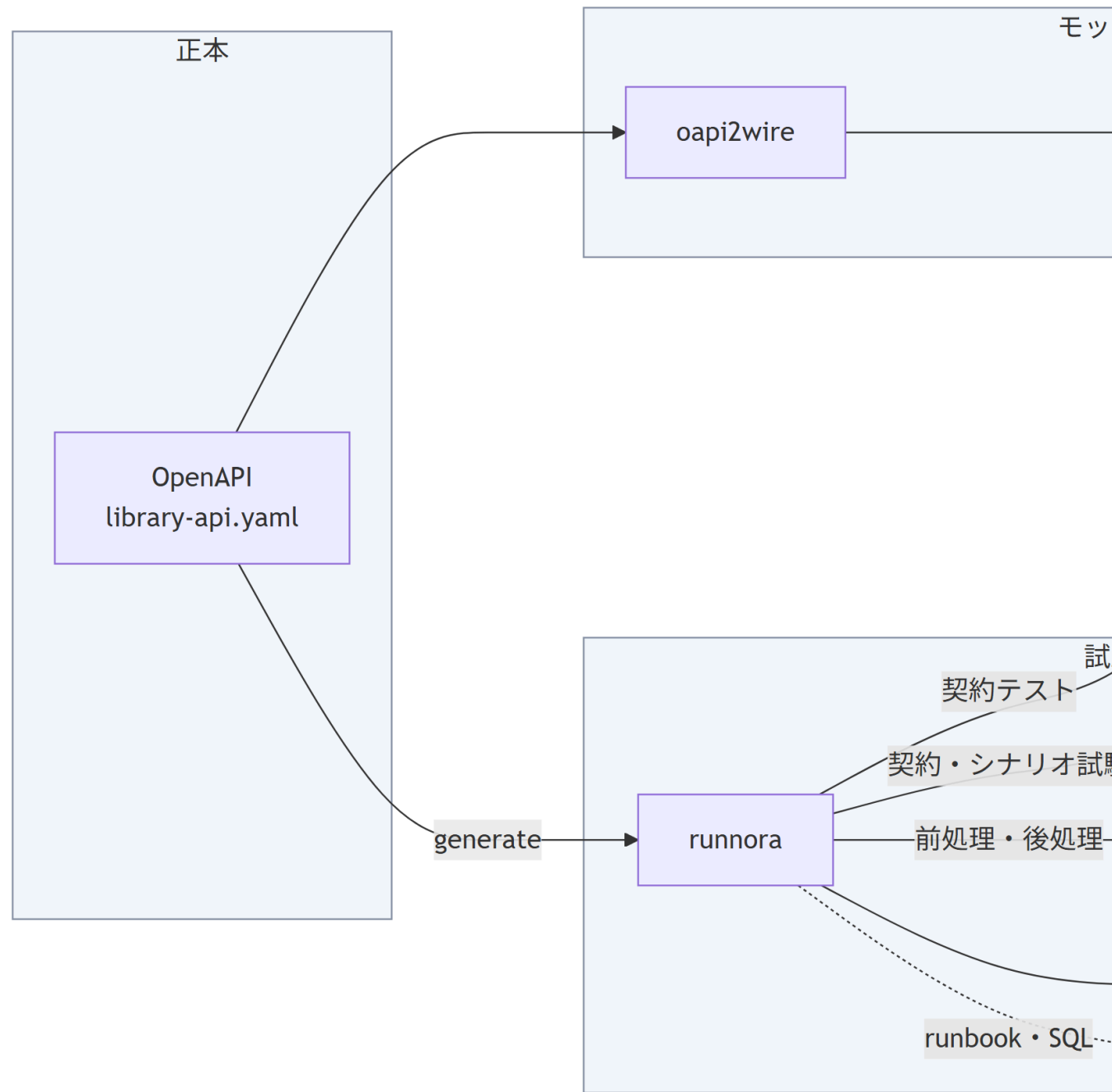


表 1.3-1 試験環境

構成要素	版・設定	備考
Oracle Database	Free 23ai (Docker container-registry.oracle.com/database/free)	ホスト 1522 → コンテナ 1521、PDB FREEPDB1
サンプル API	Go 1.24 以上、go-ora v2.9.0	127.0.0.1:18081
WireMock	3.13.2 standalone (Java 17)	127.0.0.1:18080
runnora	runn v1.9.2 組込み	前後処理は go-ora で実行 (Oracle Client 不要)

1.4 試験の種類

表 1.4-1 試験の種類

種類	実行先	資産	検証の深さ
生成テスト	モック・実装	runbooks/generated/ (runnora generate の出力そのまま)	OpenAPI の example で呼び、ステータスのみ
契約テスト	モック・実装	runbooks/contract/ , cases/contract/	ステータス + 本文の一部 + OpenAPI 応答スキーマ
シナリオ試験	実装のみ	runbooks/scenarios/ , sql/cases/	状態変化・ID 引き継ぎ・DB 直接照会・前後処理での DB 検証
検知確認	実装のみ	runbooks/demo/	事後検証が DB 不整合を検知して終了コード 4 になること

2. API 仕様

本章は `tools/openapi-doc` が `openapi/library-api.yaml` から生成した（生成）。仕様を変えるときは OpenAPI を直し、`scripts/build-docs.ps1` で再生成する。

2.1 API の概要

図書貸出 API (runnora-e2e サンプル) (版 1.0.0)

runnora / oapi2wire / runnora-docgen のフルセット検証用サンプル API。蔵書・会員・貸出を Oracle Database で管理する。

業務ルール: - 会員が同時に借りられる冊数は会員ごとの上限 (maxLoans, 既定 3) まで - 返却期限を過ぎた貸出 (延滞) がある会員は新たに借りられない - 利用停止 (SUSPENDED) の会員は借りられない - 貸出可能冊数 (availableCopies) が 0 の本は借りられない - 貸出期間は貸出日から 14 日

表 2.1-1 接続先

接続先	説明
http://127.0.0.1:18081	サンプル実装 (Go + Oracle)
http://127.0.0.1:18080	WireMock (oapi2wire 生成モック)

表 2.1-2 API 一覧

分類	メソッド	パス	operationId	概要	応答ステータス
system	GET	/health	getHealth	稼働状態を確認する	200
books	GET	/books	listBooks	蔵書を検索する	200, 400
books	POST	/books	createBook	蔵書を登録する	201, 400, 409
books	GET	/books/{bookId}	getBook	蔵書を 1 件取得する	200, 404
members	GET	/members/{memberId}	getMember	会員と貸出状況を取得する	200, 404
members	GET	/members/{memberId}/loans	listMemberLoans	会員の貸出履歴を取得する	200, 400, 404
loans	POST	/loans	createLoan	本を貸し出す	201, 400, 404, 409
loans	POST	/loans/{loanId}/return	returnLoan	本を返却する	200, 404, 409

2.2 各 API の仕様

2.2.1 GET /health (稼働状態を確認する)

表 2.2.1-1 getHealth の応答

ステータス	説明	応答スキーマ	例
200	稼働中	Health (2.3.1 節)	<code>{"database": "UP", "status": "UP"}</code>

2.2.2 GET /books (蔵書を検索する)

表 2.2.2-1 listBooks のパラメータ

名前	位置	必須	型	制約	説明	例
genre	query		string	値: NOVEL / TECH / HISTORY / TEST	ジャンルで絞り込む	
availableOnly	query		boolean		true の場合、貸出可能な本だけを返す	

表 2.2.2-2 listBooks の応答

ステータス	説明	応答スキーマ	例
200	検索結果 (book_id 昇順)	BookList (2.3.4 節)	<code>{"items": [{"author": "夏目漱石", "availableCopies": 3, "bookId": "B0001", "genre": "NOVEL", "isbn": "9784000000000", "totalCopies": 3}, {"author": "芥川龍之介", "availableCopies": 1, "bookId": "B0002", "genre": "NOVEL", "isbn": "9784000000000", "totalCopies": 1}], "total": 2}</code>
400	入力不正	Error (2.3.9 節)	<code>{"code": "VALIDATION_ERROR", "message": "memberId is required"}</code>

2.2.3 POST /books (蔵書を登録する)

リクエストボディ (application/json、必須) : BookCreate (2.3.3 節)

```
{  
  "author": "試験 花子",  
  "bookId": "B0900",  
  "genre": "TECH",  
  "isbn": "9784000000900",  
  "title": "API テスト入門",  
  "totalCopies": 2  
}
```

ステータス	説明	応答スキーマ	例
201	登録した蔵書	Book (2.3.2 節)	<code>{"author": "試験 花子", "availableCopies": 2, "bookId": "B0900", "genre": "TECH", "isbn": "9781111111111", "testEntry": "テスト入門", "totalCopies": 2}</code>
400	入力不正	Error (2.3.9 節)	<code>{"code": "VALIDATION_ERROR", "message": "member is required"}</code>
409	bookId または ISBN が登録済み	Error (2.3.9 節)	<code>{"code": "DUPLICATE_BOOK", "message": "book already exists"}</code>

表 2.2.4-1 getBook のパラメータ

名前	位置	必須	型	制約	説明	例
bookId	path	○	string	形式: <code>^B[0-9]{4}\$</code>		B0001

ステータス	説明	応答スキーマ	例
200	蔵書	Book (2.3.2 節) 石,"availableCopies":3,"bookId":"B0001","genre":"NOVEL","isbn":"9784 輩は猫である","totalCopies":3}	
404	対象が存在しない	Error (2.3.9 節)	{"code":"NOT_FOUND","message":"resource not found"}

表 2.2.5-1 getMember のパラメータ

名前	位置	必須	型	制約	説明	例
memberId	path	○	string	形式: ^M[0-9]{4}\$		M0001

ステータス	説明	応答スキーマ	例
200	会員	Member (2.3.5 節)	<code>{ "active": true, "loanCount": 0, "maxLoans": 3, "memberId": "M0001", "memberName": "田 太郎", "overdue": false, "status": "ACTIVE" }</code>
404	対象が存在しない	Error (2.3.9 節)	<code>{ "code": "NOT_FOUND", "message": "resource not found" }</code>

表 2.2.6-1 listMemberLoans のパラメータ

名前	位置	必須	型	制約	説明	例
memberId	path	○	string	形式: <code>^M[0-9]{4}\$</code>		M0001
status	query		string	値: ACTIVE / RETURNED	貸出状態で絞り込む	

表 2.2.6-2 listMemberLoans の応答

ステータス	説明	応答スキーマ	例
200	貸出一覧 (loan_id 昇順)	LoanList (2.3.8 節)	<code>{"items": [{"bookId": "B0001", "dueDate": "2026-10-10", "loanId": 1, "loanedAt": "2026-09-26", "memberId": "M0001", "returnDate": "2026-10-10"}]}</code>
400	入力不正	Error (2.3.9 節)	<code>{"code": "VALIDATION_ERROR", "message": "memberId is required"}</code>
404	対象が存在しない	Error (2.3.9 節)	<code>{"code": "NOT_FOUND", "message": "resource not found"}</code>

2.2.7 POST /loans (本を貸し出す)

リクエストボディ (application/json、必須) : LoanCreate (2.3.6 節)

```
{  
  "bookId": "B0001",  
  "memberId": "M0001"  
}
```

表 2.2.7-1 createLoan の応答

ステータス	説明	応答スキーマ	例
201	作成した貸出	Loan (2.3.7 節)	<code>{"bookId": "B0001", "dueDate": "2026-10-10", "loanId": 1, "loanedAt": "2026-09-26", "memberId": "M0001", "returnDate": "2026-10-10"}</code>
400	入力不正	Error (2.3.9 節)	<code>{"code": "VALIDATION_ERROR", "message": "memberId is required"}</code>
404	対象が存在しない	Error (2.3.9 節)	<code>{"code": "NOT_FOUND", "message": "resource not found"}</code>
409	業務ルール違反。code で理由を示す。NO_STOCK / LOAN_LIMIT_EXCEEDED / OVERDUE_LOAN / MEMBER_SUSPENDED	Error (2.3.9 節)	<code>{"code": "NO_STOCK", "message": "no available copies"}</code>

2.2.8 POST /loans/{loanId}/return (本を返却する)

表 2.2.8-1 returnLoan のパラメータ

名前	位置	必須	型	制約	説明	例
loanId	path	○	integer(int64)			1

表 2.2.8-2 returnLoan の応答

ステータス	説明	応答スキーマ	例
200	返却済みの貸出	<code>Loan</code> (2.3.7 節)	<code>{"code": "OK", "message": "loan returned", "loanId": 1, "loanedAt": "2026-09-26", "memberId": "M0001", "returnedAt": "2026-09-26"}</code>
404	対象が存在しない	Error (2.3.9 節)	<code>{"code": "NOT_FOUND", "message": "resource not found"}</code>
409	返却済み	Error (2.3.9 節)	<code>{"code": "ALREADY_RETURNED", "message": "loan already returned"}</code>

2.3 データ定義

2.3.1 Health

表 2.3.1-1 Health

項目	型	必須	制約	説明
status	string	○	値: UP / DOWN	
database	string	○	値: UP / DOWN	

2.3.2 Book

表 2.3.2-1 Book

項目	型	必須	制約	説明
bookId	string	○		
isbn	string	○		
title	string	○		
author	string	○		
genre	string	○	値: NOVEL / TECH / HISTORY / TEST	
totalCopies	integer	○		
availableCopies	integer	○		

2.3.3 BookCreate

表 2.3.3-1 BookCreate

項目	型	必須	制約	説明
bookId	string	○	形式: ^B[0-9]{4}\$	
isbn	string	○	形式: ^[0-9]{13}\$	
title	string	○	最大長 200	
author	string	○	最大長 100	
genre	string	○	値: NOVEL / TECH / HISTORY / TEST	
totalCopies	integer	○	最小 1、最大 99	

2.3.4 BookList

表 2.3.4-1 BookList

項目	型	必須	制約	説明
total	integer	○		
items	Book (2.3.2 節) の配列	○		

2.3.5 Member

表 2.3.5-1 Member

項目	型	必須	制約	説明
memberId	string	○		
name	string	○		
status	string	○	値: ACTIVE / SUSPENDED	
maxLoans	integer	○		
activeLoanCount	integer	○		
overdue	boolean	○		返却期限を過ぎた未返却の貸出があるか

2.3.6 LoanCreate

表 2.3.6-1 LoanCreate

項目	型	必須	制約	説明
memberId	string	○		
bookId	string	○		

2.3.7 Loan

表 2.3.7-1 Loan

項目	型	必須	制約	説明
loanId	integer(int64)	○		
memberId	string	○		
bookId	string	○		
loanedAt	string(date)	○		
dueDate	string(date)	○		
returnedAt	string(date)、null 可	○		
status	string	○	値: ACTIVE / RETURNED	

2.3.8 LoanList

表 2.3.8-1 LoanList

項目	型	必須	制約	説明
total	integer	○		
items	Loan (2.3.7 節) の配列	○		

2.3.9 Error

表 2.3.9-1 Error

項目	型	必須	制約	説明
code	string	○		
message	string	○		

3. 試験の設計

3.1 資産の流れ

OpenAPI を唯一の正本とし、そこから次の順で資産を作る。人が書くのは太字の部分だけである。

表 3.1-1 資産の流れ

順	作業	ツール	入力	出力
1	API 仕様を書く	-	-	openapi/library-api.yaml
2	モックの雛形を作る	oapi2wire init	OpenAPI	mock/mock-cases.yaml, mock/mock-responses/
3	返し分けを書く	-	雛形	mock/mock-cases.yaml, mock/mock-responses/
4	モックを生成する	oapi2wire validate / build	OpenAPI + case YAML	mock/wiremock-out/
5	テストの雛形を生成する	runnora generate	OpenAPI	runbooks/generated/, cases/generated/
6	契約テストを書く	-	生成 template を複製	runbooks/contract/, cases/contract/
7	シナリオと前後処理を書く	-	-	runbooks/scenarios/, sql/
8	手順書を生成する	runnora-docgen, tools/openapi-doc, ddq	上記すべて	docs/generated/, 本書

モックと実装が同じ契約テストに合格するよう、次の値を 3 か所でそろえている。

- OpenAPI の **example** (生成テストの入力・期待値になる)
- モックの返し分け条件と応答 JSON (**mock/**)
- 実 DB の初期データ (**sql/common/10_seed_master.sql**)

3.2 初期データ

runbook を実行するたびに、共通前処理がデータを全削除して次の状態に戻す（3.3 節）。貸出 ID の採番も 1 に戻すため、シナリオは貸出 ID を固定値で検証できる。

表 3.2-1 初期データ

区分	ID	内容
会員	M0001 山田 太郎	ACTIVE、上限 3
会員	M0002 佐藤 花子	ACTIVE、上限 3
会員	M0003 鈴木 一郎	SUSPENDED （利用停止）
会員	M0004～M0010	テスト会員、ACTIVE、上限 3
蔵書	B0001 吾輩は猫である	NOVEL、3 冊
蔵書	B0002 羅生門	NOVEL、 1 冊 （在庫切れの確認用）
蔵書	B0003 Go 言語による Web アプリ開発	TECH、2 冊
蔵書	B0004 日本の城郭史	HISTORY、1 冊
蔵書	B0005～B0020	テスト図書。偶数は TECH・奇数は HISTORY、B0010 までは 1 冊・以降 2 冊

3.3 PL/SQL 前後処理

3.3.1 実行順序

runnora は runbook 1 本ごとに次の順で処理する。後処理は runbook が失敗しても実行される。suite が `include + loop` で template を繰り返しても、前後処理は suite 1 本につき 1 回である。

表 3.3.1-1 前後処理の実行順序

順	区分	ファイル	役割
1	共通前処理	sql/common/00_reset.sql	全データ削除、貸出 ID 採番のリセット
2	共通前処理	sql/common/10_seed_master.sql	会員・蔵書マスタの投入
3	ケース前処理	--before-sql (sql/cases/*_before.sql 等)	API では作れない状態（延滞など）を作る
4	runbook	runbooks/**	API 呼び出しと検証
5	ケース後処理	--after-sql (sql/cases/*_after.sql)	ケース固有の事後条件を DB で検証
6	共通後処理	sql/common/90_verify_integrity.sql	DB 不変条件を検証。違反は RAISE_APPLICATION_ERROR

後処理の SQL が `RAISE_APPLICATION_ERROR` で失敗すると、runnora は終了コード 4（フック失敗）で終わる。これにより「API の応答は正しいが DB が壊れている」不具合を検出できる（検知確認 DEMO で確認）。

3.3.2 記述上の制約

runnora はフックファイルの内容全体を 1 つの文として実行する。作成時に次を実機で確認した。

表 3.3.2-1 フック SQL の記述上の制約

書き方	可否	確認結果
1 ファイル = 1 つの無名ブロック (<code>DECLARE ... BEGIN ... END;</code>)	○	複数の DML/DDL はブロック内に書く
先頭・行末の <code>--</code> コメント、日本語コメント	○	そのまま実行できる
SQL*Plus の / 終端	×	PLS-00103: Encountered the symbol "/" で終了コード 4
DDL (<code>ALTER SEQUENCE</code> など)	○	<code>EXECUTE IMMEDIATE</code> で実行する。暗黙コミットに注意
ブロック内の局所ファンクションを SQL 文から呼ぶ	×	PLS-00231。一度変数に代入してから使う
<code>TIMESTAMP</code> (TZ なし) 列と <code>SYSTIMESTAMP</code> の比較	△	比較は TZ なし側をセッション TZ で解釈するため (go-ora 接続で 9 時間ずれて失敗した) 結果が環境依存。 <code>TIMESTAMP WITH TIME ZONE</code> を使う
ファイルをまたぐ <code>SAVEPOINT</code> ・トランザクション	×	ファイルごとに接続プールから実行されるため、各ファイル内で完結させる

3.3.3 使用している PL/SQL 構文

表 3.3.3-1 使用している PL/SQL 構文

構文	使用ファイル	目的
数値 FOR LOOP	00_reset, 10_seed_master	表ごとの削除、テスト会員・図書の連番生成
カーソル FOR LOOP	90_verify_integrity	蔵書ごと・貸出ごとの不変条件検証
明示カーソル（パラメータ付き、FOR UPDATE、WHERE CURRENT OF） + WHILE LOOP	lib004_fill_loans_before	上限に達するまで貸出を作る
BULK COLLECT ... LIMIT + FORALL + SQL%BULK_ROWCOUNT	lib007_history_before	貸出履歴 16 件を 5 件ずつ一括投入
SAVEPOINT / ROLLBACK TO SAVEPOINT	lib005_savepoint_before	失敗した単位だけを取り消し、他は確定
例外処理（DUP_VAL_ON_INDEX、NO_DATA_FOUND、TOO_MANY_ROWS、ユーザ定義例外）	lib005, lib001_assert_after	想定内の失敗の分岐、事後条件の判定
PRAGMA EXCEPTION_INIT	lib005_savepoint_before	ORA-02291（親キーなし）に名前を付ける
PRAGMA AUTONOMOUS_TRANSACTION	00_reset, 90_verify, lib005, lib007	ROLLBACK されないフック実行ログ
EXECUTE IMMEDIATE（DML・DDL）、DBMS_ASSERT	00_reset	動的な表名の削除、シーケンスのリセット
MERGE	10_seed_master	会員マスタの登録・更新
レコード型・連想配列・VARRAY・入れ子表・修飾式	00_reset, 10_seed, lib005, lib007	テストデータの表形式定義
%ROWTYPE とレコード単位の INSERT	lib003_overdue_before, lib001_assert_after	貸出行の組み立て・取得
RETURNING INTO	contract_setup, lib003, lib005	採番結果の取得
RAISE_APPLICATION_ERROR	90_verify_integrity, lib00x_assert_after	事後条件違反を runnora に伝える（終了コード 4）

3.4 モックの返し分け

WireMock は状態を持たないため、返し分けは「入力 → 固定応答」の対応で定義する。各 operation に fallback を置き、想定外の入力には 404 / 400 を返す（未定義の組合せを 501 にしない）。

表 3.4-1 モックの返し分け (mock/mock-cases.yaml)

operation	条件	応答
listBooks	genre=NOVEL / genre=TECH & availableOnly=true / 定義外 genre / その他	200 / 200 / 400 / 200 (全件)
getBook	B0001・B0002 / その他	200 / 404
createBook	B0900 の登録 / ISBN 9784000000001 / その他	201 / 409 / 400
getMember	M0001・M0003 / その他	200 / 404
listMemberLoans	M0001・M0002 / その他	200 / 404
createLoan	M0001+B0001 / memberId=M0003 / bookId=B9999 / その他	201 / 409 / 404 / 400
returnLoan	loanId=1 / loanId=2 / その他	200 / 409 / 404

4. 実施手順

4.1 準備

表 4.1-1 準備

No	作業	コマンド（リポジトリ直下、PowerShell）	確認
1	Docker Desktop を起動する	-	<code>docker info</code> が応答する
2	Oracle を起動しスキーマを作る	<code>./scripts/db-up.ps1</code>	「LIBAPP スキーマを作成しました」と表示
3	モック資産を生成する	<code>./scripts/mock-build.ps1</code>	<code>build complete</code> と表示
4	テスト雛形を再生成する（任意）	<code>./scripts/generate.ps1</code>	<code>Generated 8 operations</code> と表示

4.2 実行

一括で実行する場合は `./scripts/run-all.ps1` を使う。API と WireMock をバックグラウンドで起動し、テスト・手順書生成の後に停止する。個別に実行する場合は次のとおり。

表 4.2-1 実行

No.	作業	コマンド	合格条件
1	API を起動する（ターミナル A）	<code>./scripts/api-start.ps1</code>	<code>library api listening</code> と表示
2	モックを起動する（ターミナル B）	<code>./scripts/mock-start.ps1</code>	18080 で待ち受け
3	モックに対してテストする	<code>./scripts/test-mock.ps1</code>	「モック: 2/2 OK」
4	実 API に対してテストする	<code>./scripts/test-api.ps1</code>	「実 API: 10/10 OK」 カバレッジ 8/8
5	手順書を生成する	<code>./scripts/build-docs.ps1</code>	<code>docs/_book/index.html</code> ができる

`test-api.ps1` はシナリオごとに次の形で `runnora` を呼ぶ（`scripts/scenarios.psd1` の定義による）。

```
runnora run --config config.yaml `
  --before-sql sql/cases/lib005_savepoint_before.sql `
  --after-sql sql/cases/lib005_assert_after.sql `
  runbooks/scenarios/lib-005-savepoint.yml
```

終了コードの意味は次のとおり。検知確認（DEMO）は 4 が合格である。

表 4.2-2 runnora の終了コード

終了コード	意味
0	全成功
1	runbook の検証失敗
2	設定・引数不正
3	DB 接続失敗
4	前処理・後処理の失敗（事後検証の違反を含む）

4.3 証跡

reports/<日時>-mock/、reports/<日時>-api/ に、実行ごとの結果 (*.txt)、標準出力 (*.log)、カバレッジ (coverage.txt) が残る。DB 側の証跡として、前後処理のログが TEST_HOOK_LOG 表に残る (1 日で自動削除)。

4.4 終了

表 4.4-1 終了

No.	作業	コマンド
1	API・モックを止める	各ターミナルで Ctrl+C
2	Oracle を止めて削除する	<code>./scripts/db-down.ps1</code>

5. 契約テスト

契約テストは、同じケース JSON を**モック（WireMock）と実装（実 API）の両方**に流し、どちらも OpenAPI どおりに応答することを確認する。各 suite は `runnora generate` が作った template を `runbooks/contract/` に複製し、次の 2 点を追加したものである。

- 検証式にケース JSON の `expect.body` のトップレベル項目との一致（部分一致）を追加
- ランナーに `openapi3` を指定し、応答を OpenAPI の応答定義・スキーマで自動検証（作成時、`GET /books` などの 400 応答が OpenAPI に未定義であることをこの検証で検出し、定義を補った）

表 5-1 契約テストの実行先

実行先	設定	前処理	実行コマンド
モック	<code>config.mock.yaml</code> （DB なし）	なし	<code>./scripts/test-mock.ps1</code>
実 API	<code>config.yaml</code>	共通前処理 + <code>sql/cases/contract_setup.sql</code> （貸出 1: 貸出中、貸出 2: 返却済）	<code>./scripts/test-api.ps1</code>

合格条件：両方の実行先で 8 suite すべてが成功すること。

以下の表は `runnora-docgen` が生成した（生成）。前処理・後処理の表は実 API で実行する場合の内容である。

5.1 前処理・後処理（全 suite 共通）

表 5.1-1 前処理（契約 `getBook` 蔵書を 1 件取得する（モック・実 API の両方で実行する））

順序	ファイル	出典
1	<code>00_reset.sql</code>	<code>sql/common/00_reset.sql</code>
2	<code>10_seed_master.sql</code>	<code>sql/common/10_seed_master.sql</code>
3	<code>contract_setup.sql</code>	<code>sql/cases/contract_setup.sql</code>

表 5.1-2 後処理（契約 `getBook` 蔵書を 1 件取得する（モック・実 API の両方で実行する））

順序	ファイル	出典
1	<code>90_verify_integrity.sql</code>	<code>sql/common/90_verify_integrity.sql</code>

5.2 契約 getBook 蔵書を 1 件取得する（モック・実 API の両方で実行する）

出典：runbooks/contract/get_getBook.suite.yml

5.2.1 テストシナリオ

表 5.2.1-1 テストシナリオ

シナリオ	契約 getBook 蔵書を 1 件取得する（モック・実 API の両方で実行する）			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL : 表 5.2.3-1-[1]		期待値 : [ケース別] 表 5.2.4-1 01_B0001.json 02_not_found.json	

5.2.2 ケースデータ

表 5.2.2-1 ケースデータ一覧（契約 getBook 蔵書を 1 件取得する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
B0001	登録済みの蔵書 B0001	pathParams: {"bookId":"B0001"}	表 5.2.4-1	01_B0001.json
not_found	存在しない蔵書は 404	pathParams: {"bookId":"B9999"}	表 5.2.4-1	02_not_found.json

5.2.3 HTTP 呼び出し

表 5.2.3-1 HTTP 呼び出し情報（契約 getBook 蔵書を 1 件取得する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	GET	\${RUNNORA_BASE_URL}/books/ { { vars.case.pathParams.book Id } }	"{ { vars.case.headers } }"	

5.2.4 期待値・検証条件

表 5.2.4-1 期待値・検証条件（契約 getBook 蔵書を 1 件取得する（モック・実 API の両方で実行する）／手順 1）

区分	出典	内容
検証式	get_getBook .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP ケース別
ケース B0001	01_B0001.json ケース定義（検証式参照未 確認）	\$.body.author: 夏目漱石
		\$.body.availableCopies: 3
		\$.body.bookId: B0001
		\$.body.genre: NOVEL
		\$.body.isbn: 9784000000001
		\$.body.title: 吾輩は猫である
		\$.body.totalCopies: 3
		\$.bodyMode: subset
		\$.ignorePaths: []
	01_B0001.json ケース定義（検証式参照）	\$.status: 200
ケース not_found	02_not_found.json ケース定義（検証式参照未 確認）	\$.body.code: NOT_FOUND
		\$.bodyMode: subset
	02_not_found.json ケース定義（検証式参照）	\$.ignorePaths: []
	02_not_found.json ケース定義（検証式参照）	\$.status: 404

5.3 契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する）

出典：runbooks/contract/get_getHealth.suite.yml

5.3.1 テストシナリオ

表 5.3.1-1 テストシナリオ

シナリオ	契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する）			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL：表 5.3.3-1-[1]		期待値：[200] 表 5.3.4-1 01_up.json	

5.3.2 ケースデータ

表 5.3.2-1 ケースデータ一覧（契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
up	API と DB が稼働中		表 5.3.4-1	01_up.json

5.3.3 HTTP 呼び出し

表 5.3.3-1 HTTP 呼び出し情報（契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	GET	\${RUNNORA_BASE_URL}/health	"{{ vars.case.headers }}"	

5.3.4 期待値・検証条件

表 5.3.4-1 期待値・検証条件（契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する）／手順 1）（1／2）

区分	出典	内容
検証式	get_getHealth .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP 200
ケース up		\$.body.database: UP
		\$.body.status: UP

表 5.3.4-1 期待値・検証条件（契約 getHealth 稼働状態を確認する（モック・実 API の両方で実行する）／手順 1）（2／2）

区分	出典	内容
	01_up.json	\$.bodyMode: subset
	ケース定義（検証式参照未 確認）	\$.ignorePaths: []
	01_up.json	\$.status: 200
	ケース定義（検証式参照）	

5.4 契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する）

出典：runbooks/contract/get_getMember.suite.yml

5.4.1 テストシナリオ

表 5.4.1-1 テストシナリオ

シナリオ	契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する）			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL：表 5.4.3-1-[1]		期待値：[ケース別] 表 5.4.4-1 01_M0001.json 02_M0003_suspended.json 03_not_found.json	

5.4.2 ケースデータ

表 5.4.2-1 ケースデータ一覧（契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
M0001	貸出のない会員 M0001	pathParams: { "memberId": "M0001" }	表 5.4.4-1	01_M0001.json
M0003_suspended	利用停止中の会員 M0003	pathParams: { "memberId": "M0003" }	表 5.4.4-1	02_M0003_suspended.json
not_found	存在しない会員は 404	pathParams: { "memberId": "M9999" }	表 5.4.4-1	03_not_found.json

5.4.3 HTTP 呼び出し

表 5.4.3-1 HTTP 呼び出し情報（契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	GET	\${RUNNORA_BASE_URL}/members/ { { vars.case.pathParams.memberId } }	"{ { vars.case.headers } }"	

5.4.4 期待値・検証条件

表 5.4.4-1 期待値・検証条件（契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する）／手順 1）（1／2）

区分	出典	内容
検証式	get_getMember .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP ケース別
ケース M0001	01_M0001.json ケース定義（検証式参照未確認）	\$.body.activeLoanCount: 0
		\$.body.maxLoans: 3
		\$.body.memberId: M0001
		\$.body.name: 山田 太郎
		\$.body.overdue: false
		\$.body.status: ACTIVE
		\$.bodyMode: subset
		\$.ignorePaths: []
	01_M0001.json ケース定義（検証式参照）	\$.status: 200
ケース M0003_suspended	02_M0003_suspended.json ケース定義（検証式参照未確認）	\$.body.memberId: M0003
		\$.body.status: SUSPENDED
		\$.bodyMode: subset
	02_M0003_suspended.json ケース定義（検証式参照）	\$.status: 200
ケース not_found	03_not_found.json ケース定義（検証式参照未確認）	\$.body.code: NOT_FOUND
		\$.bodyMode: subset
		\$.ignorePaths: []

表 5.4.4-1 期待値・検証条件（契約 getMember 会員と貸出状況を取得する（モック・実 API の両方で実行する）／手順 1）（2／2）

区分	出典	内容
	03_not_found.json ケース定義（検証式参照）	\$.status: 404

5.5 契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する）

出典：runbooks/contract/get_listBooks.suite.yml

5.5.1 テストシナリオ

表 5.5.1-1 テストシナリオ

シナリオ	契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する）			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL : 表 5.5.3-1-[1]		期待値 : [ケース別] 表 5.5.4-1 01_novel.json 02_tech_available.json 03_invalid_genre.json	

5.5.2 ケースデータ

表 5.5.2-1 ケースデータ一覧（契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
novel	小説の検索は B0001・B0002 の 2 件	queryParams: { "availableOnly": "", genre": "NOVEL" }	表 5.5.4-1	01_novel.json
tech_available	貸出可能な技術書の検索は 200	queryParams: { "availableOnly": "true", genre": "TECH" }	表 5.5.4-1	02_tech_available.json
invalid_genre	定義外のジャンルは 400	queryParams: { "availableOnly": "", genre": "COMIC" }	表 5.5.4-1	03_invalid_genre.json

5.5.3 HTTP 呼び出し

表 5.5.3-1 HTTP 呼び出し情報（契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	GET	<code>\${RUNNORA_BASE_URL}/books?</code> <code>genre={{ vars.case.queryParams.genre }}</code> <code>&availableOnly={{ vars.case.queryParams.availableOnly }}</code>	<code>"{{ vars.case.headers }}"</code>	

5.5.4 期待値・検証条件

表 5.5.4-1 期待値・検証条件（契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する）／手順 1）（1／2）

区分	出典	内容
検証式	get_listBooks .template.yml:24	<code>current.res.status == vars.case.expect.status &&</code> <code>all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })</code>
ステータス	検証式から抽出	HTTP ケース別
ケース novel	01_novel.json ケース定義（検証式参照未 確認）	<code>\$.body.items[0].author: 夏目漱石</code>
		<code>\$.body.items[0].availableCopies: 3</code>
		<code>\$.body.items[0].bookId: B0001</code>
		<code>\$.body.items[0].genre: NOVEL</code>
		<code>\$.body.items[0].isbn: 9784000000001</code>
		<code>\$.body.items[0].title: 吾輩は猫である</code>
		<code>\$.body.items[0].totalCopies: 3</code>
		<code>\$.body.items[1].author: 芥川龍之介</code>
		<code>\$.body.items[1].availableCopies: 1</code>
		<code>\$.body.items[1].bookId: B0002</code>
		<code>\$.body.items[1].genre: NOVEL</code>
		<code>\$.body.items[1].isbn: 9784000000002</code>
		<code>\$.body.items[1].title: 羅生門</code>

表 5.5.4-1 期待値・検証条件（契約 listBooks 蔵書を検索する（モック・実 API の両方で実行する）／手順 1）（2／2）

区分	出典	内容
		\$.body.items[1].totalCopies: 1
		\$.body.total: 2
		\$.bodyMode: subset
		\$.ignorePaths: []
	01_novel.json ケース定義（検証式参照）	\$.status: 200
ケース tech_available	02_tech_available.json ケース定義（検証式参照）	\$.body: {}
	02_tech_available.json ケース定義（検証式参照未確認）	\$.bodyMode: subset
		\$.ignorePaths: []
	02_tech_available.json ケース定義（検証式参照）	\$.status: 200
ケース invalid_genre	03_invalid_genre.json ケース定義（検証式参照未確認）	\$.body.code: VALIDATION_ERROR
		\$.bodyMode: subset
		\$.ignorePaths: []
	03_invalid_genre.json ケース定義（検証式参照）	\$.status: 400

5.6 契約 listMemberLoans 会員の貸出履歴を取得する（モック・実 API の両方で実行する）

出典：runbooks/contract/get_listMemberLoans.suite.yml

5.6.1 テストシナリオ

表 5.6.1-1 テストシナリオ

シナリオ	契約 listMemberLoans 会員の貸出履歴を取得する（モック・実 API の両方で実行する）			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL：表 5.6.3-1-[1]		期待値：[ケース別] 表 5.6.4-1 01_M0002.json 02_not_found.json	

5.6.2 ケースデータ

表 5.6.2-1 ケースデータ一覧（契約 listMemberLoans 会員の貸出履歴を取得する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
M0002	M0002 は貸出中 1 件・返却済 1 件 (contract_setup.sql)	pathParams: {"memberId":"M0002"} queryParams: {"status":""}	表 5.6.4-1	01_M0002.json
not_found	存在しない会員は 404	pathParams: {"memberId":"M9999"} queryParams: {"status":""}	表 5.6.4-1	02_not_found.json

5.6.3 HTTP 呼び出し

表 5.6.3-1 HTTP 呼び出し情報（契約 listMemberLoans 会員の貸出履歴を取得する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	GET	\${RUNNORA_BASE_URL}/members/ { vars.case.pathParams.memberId }/loans? status={ vars.case.queryParams.status }	"{{ vars.case.headers }}"	

5.6.4 期待値・検証条件

表 5.6.4-1 期待値・検証条件（契約 listMemberLoans 会員の貸出履歴を取得する（モック・実 API の両方で実行する）／手順 1）

区分	出典	内容
検証式	get_ listMember Loans .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP ケース別
ケース M0002	01_M0002.json ケース定義（検証式参照未 確認）	\$.body.total: 2
		\$.bodyMode: subset
		\$.ignorePaths: []
	01_M0002.json ケース定義（検証式参照）	\$.status: 200
ケース not_found	02_not_found.json ケース定義（検証式参照未 確認）	\$.body.code: NOT_FOUND
		\$.bodyMode: subset
		\$.ignorePaths: []
	02_not_found.json ケース定義（検証式参照）	\$.status: 404

5.7 契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）

出典：runbooks/contract/post_createBook.suite.yml

5.7.1 テストシナリオ

表 5.7.1-1 テストシナリオ

シナリオ	契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL：表 5.7.3-1-[1]	表 5.7.4-1 01_created.json 02_duplicate_isbn.json 03_invalid_isbn.json	期待値：[ケース別] 表 5.7.5-1 01_created.json 02_duplicate_isbn.json 03_invalid_isbn.json	

5.7.2 ケースデータ

表 5.7.2-1 ケースデータ一覧（契約 createBook 蔵書を登録する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
created	新しい蔵書 B0900 を登録する	requestBody: 表 5.7.4-1	表 5.7.5-1	01_created.json
duplicate_isbn	B0001 と同じ ISBN は 409	requestBody: 表 5.7.4-1	表 5.7.5-1	02_duplicate_isbn.json
invalid_isbn	13 桁でない ISBN は 400	requestBody: 表 5.7.4-1	表 5.7.5-1	03_invalid_isbn.json

5.7.3 HTTP 呼び出し

表 5.7.3-1 HTTP 呼び出し情報（契約 createBook 蔵書を登録する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	POST	\${RUNNORA_BASE_URL}/books	"{{ vars.case.headers }}"	

5.7.4 リクエストボディ

表 5.7.4-1 リクエストボディ（契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）／手順 1）（1／2）

ケース	JSON path	値
created	\$.author	試験 花子

表 5.7.4-1 リクエストボディ（契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）／手順 1）（2／2）

ケース	JSON path	値
	\$.bookId	B0900
	\$.genre	TECH
	\$.isbn	9784000000900
	\$.title	API テスト入門
	\$.totalCopies	2
duplicate_isbn	\$.author	試験 花子
	\$.bookId	B0901
	\$.genre	TECH
	\$.isbn	9784000000001
	\$.title	重複 ISBN
	\$.totalCopies	1
invalid_isbn	\$.author	試験 花子
	\$.bookId	B0902
	\$.genre	TECH
	\$.isbn	978400
	\$.title	不正 ISBN
	\$.totalCopies	1

5.7.5 期待値・検証条件

表 5.7.5-1 期待値・検証条件（契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）／手順 1）（1／2）

区分	出典	内容
検証式	post_createBook .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP ケース別

表 5.7.5-1 期待値・検証条件（契約 createBook 蔵書を登録する（モック・実 API の両方で実行する）／手順 1）（2／2）

区分	出典	内容
ケース created	01_created.json ケース定義（検証式参照未確認）	\$.body.availableCopies: 2
		\$.body.bookId: B0900
		\$.body.totalCopies: 2
		\$.bodyMode: subset
	01_created.json ケース定義（検証式参照）	\$.ignorePaths: []
ケース duplicate_i sbn	02_duplicate_isbn.json ケース定義（検証式参照未確認）	\$.body.code: DUPLICATE_BOOK
		\$.bodyMode: subset
		\$.ignorePaths: []
	02_duplicate_isbn.json ケース定義（検証式参照）	\$.status: 409
ケース invalid_isb n	03_invalid_isbn.json ケース定義（検証式参照未確認）	\$.body.code: VALIDATION_ERROR
		\$.bodyMode: subset
		\$.ignorePaths: []
	03_invalid_isbn.json ケース定義（検証式参照）	\$.status: 400

5.8 契約 createLoan 本を貸し出す（モック・実 API の両方で実行する）

出典：runbooks/contract/post_createLoan.suite.yml

5.8.1 テストシナリオ

表 5.8.1-1 テストシナリオ

シナリオ	契約 createLoan 本を貸し出す（モック・実 API の両方で実行する）			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL：表 5.8.3-1-[1]	表 5.8.4-1 01_created.json 02_member_suspended.js on 03_book_not_found.js n 04_missing_book_id.js on	期待値：[ケース別] 表 5.8.5-1 01_created.json 02_member_suspended.json 03_book_not_found.json 04_missing_book_id.json	

5.8.2 ケースデータ

表 5.8.2-1 ケースデータ一覧（契約 createLoan 本を貸し出す（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
created	M0001 が B0001 を借りる	requestBody: 表 5.8.4-1	表 5.8.5-1	01_created.json
member_suspended	利用停止中の M0003 は 409	requestBody: 表 5.8.4-1	表 5.8.5-1	02_member_suspended.json
book_not_found	存在しない蔵書は 404	requestBody: 表 5.8.4-1	表 5.8.5-1	03_book_not_found.json
missing_book_id	bookId なしは 400	requestBody: 表 5.8.4-1	表 5.8.5-1	04_missing_book_id.json

5.8.3 HTTP 呼び出し

表 5.8.3-1 HTTP 呼び出し情報（契約 createLoan 本を貸し出す（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	POST	\${RUNNORA_BASE_URL}/loans	"{{ vars.case.headers }}"	

5.8.4 リクエストボディ

表 5.8.4-1 リクエストボディ（契約 createLoan 本を貸し出す（モック・実 API の両方で実行する）／手順 1）

ケース	JSON path	値
created	\$.bookId	B0001
	\$.memberId	M0001
member_suspended	\$.bookId	B0001
	\$.memberId	M0003
book_not_found	\$.bookId	B9999
	\$.memberId	M0001
missing_book_id	\$.memberId	M0001

5.8.5 期待値・検証条件

表 5.8.5-1 期待値・検証条件（契約 createLoan 本を貸し出す（モック・実 API の両方で実行する）／手順 1）（1／2）

区分	出典	内容
検証式	post_createLoan .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP ケース別
ケース created	01_created.json ケース定義（検証式参照未 確認）	\$.body.bookId: B0001
		\$.body.memberId: M0001
		\$.body.returnedAt:
		\$.body.status: ACTIVE
		\$.bodyMode: subset
		\$.ignorePaths: []
	01_created.json ケース定義（検証式参照）	\$.status: 201
		\$.body.code: MEMBER_SUSPENDED

表 5.8.5-1 期待値・検証条件（契約 createLoan 本を貸し出す（モック・実 API の両方で実行する）／手順 1）（2／2）

区分	出典	内容
ケース member_suspended	02_member_suspended.json	\$.bodyMode: subset
	ケース定義（検証式参照未確認）	\$.ignorePaths: []
	02_member_suspended.json	\$.status: 409
ケース book_not_found	03_book_not_found.json	\$.body.code: NOT_FOUND
	ケース定義（検証式参照未確認）	\$.bodyMode: subset
		\$.ignorePaths: []
	03_book_not_found.json	\$.status: 404
	ケース定義（検証式参照）	
ケース missing_book_id	04_missing_book_id.json	\$.body.code: VALIDATION_ERROR
	ケース定義（検証式参照未確認）	\$.bodyMode: subset
		\$.ignorePaths: []
	04_missing_book_id.json	\$.status: 400
	ケース定義（検証式参照）	

5.9 契約 returnLoan 本を返却する（モック・実 API の両方で実行する）

出典：runbooks/contract/post_returnLoan.suite.yml

5.9.1 テストシナリオ

表 5.9.1-1 テストシナリオ

シナリオ	契約 returnLoan 本を返却する（モック・実 API の両方で実行する）			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	call_api URL：表 5.9.3-1-[1]	表 5.9.4-1 01_returned.json 02_already_returned.js on 03_not_found.json	期待値：[ケース別] 表 5.9.5-1 01_returned.json 02_already_returned.json 03_not_found.json	

5.9.2 ケースデータ

表 5.9.2-1 ケースデータ一覧（契約 returnLoan 本を返却する（モック・実 API の両方で実行する））

ケース	説明	入力	期待値	出典
returned	貸出中の貸出 1 を返却する (contract_setup.sql)	pathParams: {"loanId":1}	表 5.9.5-1	01_returned.json
already_returned	返却済みの貸出 2 は 409	pathParams: {"loanId":2}	表 5.9.5-1	02_already_returned.json
not_found	存在しない貸出は 404	pathParams: {"loanId":9999}	表 5.9.5-1	03_not_found.json

5.9.3 HTTP 呼び出し

表 5.9.3-1 HTTP 呼び出し情報（契約 returnLoan 本を返却する（モック・実 API の両方で実行する））

手順	Method	URL	Headers	Query
1	POST	\${RUNNORA_BASE_URL}/loans/ {{ vars.case.pathParams.loan Id }}/return	"{{ vars.case.headers }}"	

5.9.4 リクエストボディ

表 5.9.4-1 リクエストボディ（契約 returnLoan 本を返却する（モック・実 API の両方で実行する）／手順 1）

ケース	JSON path	値
returned	\$	{}
already_returned	\$	{}
not_found	\$	{}

5.9.5 期待値・検証条件

表 5.9.5-1 期待値・検証条件（契約 returnLoan 本を返却する（モック・実 API の両方で実行する）／手順 1）（1／2）

区分	出典	内容
検証式	post_returnLoan .template.yml:24	current.res.status == vars.case.expect.status && all(keys(vars.case.expect.body), { current.res.body[#] == vars.case.expect.body[#] })
ステータス	検証式から抽出	HTTP ケース別
ケース returned	01_returned.json ケース定義（検証式参照未 確認）	\$.body.bookId: B0003
		\$.body.loanId: 1
		\$.body.memberId: M0002
		\$.body.status: RETURNED
		\$.bodyMode: subset
		\$.ignorePaths: []
ケース already_re turned	01_returned.json ケース定義（検証式参照）	\$.status: 200
	02_already_returned.json ケース定義（検証式参照未 確認）	\$.status: 409
	02_already_returned.json ケース定義（検証式参照未 確認）	\$.body.code: ALREADY_RETURNED
		\$.bodyMode: subset
		\$.ignorePaths: []
	02_already_returned.json ケース定義（検証式参照）	

表 5.9.5-1 期待値・検証条件（契約 returnLoan 本を返却する（モック・実 API の両方で実行する）／手順 1）（2／2）

区分	出典	内容
ケース not_found	03_not_found.json	\$.body.code: NOT_FOUND
	ケース定義（検証式参照未 確認）	\$.bodyMode: subset
		\$.ignorePaths: []
	03_not_found.json ケース定義（検証式参照）	\$.status: 404

6. シナリオ試験

実装（実 API + Oracle）に対し、業務の流れに沿って状態の変化を追う。各シナリオは runbook 1 本と、必要に応じてケース固有の前処理・後処理 SQL で構成する。

表 6-1 シナリオ一覧

ID	シナリオ	ケース前処理	ケース後処理	主な確認点
LIB-001	貸出～返却	-	lib001_assert_after	ID 引き継ぎ、在庫増減、二重返却、DB 直接照会
LIB-002	在庫切れ	-	-	1 冊の本の取り合い、返却後の再貸出
LIB-003	延滞	lib003_overdue_before	-	API では作れない延滞状態を PL/SQL で作る
LIB-004	貸出上限	lib004_fill_loans_before	-	明示カーソル + WHILE で上限まで貸し出す
LIB-005	SAVEPOINT	lib005_savepoint_before	lib005_assert_after	一部だけ確定した初期データ、採番の欠番
LIB-006	異常系	-	lib006_assert_no_loans_after	400/404/409 の後に DB が変わっていない
LIB-007	大量履歴	lib007_history_before	-	BULK COLLECT/FORALL で投入した履歴の照会
DEMO	検知確認	-	demo_break_invariant_after	事後検証が不整合を検知して終了コード 4

すべてのシナリオで、共通前処理（データ初期化）と共通後処理（不変条件の検証）が前後に実行される。以下、各節の表は runnora-docgen が runbook と前後処理 SQL から生成した（生成）。DB 照会ステップは検証式だけが表示される（SQL 本文は runbook を参照）。

6.1 LIB-001 貸出～返却の正常系

- 目的：貸出と返却で在庫・会員の貸出件数が正しく増減し、貸出 ID を後続手順へ引き継げることを確認する。
- 前提：共通前処理のみ（B0001 は 3 冊、M0001 は貸出 0 件）。
- 合格条件：全手順の検証式が真であり、後処理 `lib001_assert_after.sql` が DB 上で「貸出 1 件・返却済・返却日が本日・B0001 の貸出可能冊数 3」を確認して正常終了すること。
- 実行：`runnora run --config config.yaml --after-sql sql/cases/lib001_assert_after.sql runbooks/scenarios/lib-001-loan-lifecycle.yml`

6.1.1 テストシナリオ

表 6.1.1-1 テストシナリオ (1/2)

シナリオ	LIB-001 本を借りて返すまでの正常系 (在庫増減・ID 引き継ぎ・二重返却)			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	会員 M0001 が貸出 0 件・延滞なしであることを確認する URL : 表 6.1.4-1-[1]		期待値 : [200] 表 6.1.6-1	
2	蔵書 B0001 の貸出可能冊数が所蔵冊数 3 と同じであることを確認する URL : 表 6.1.4-1-[2]		期待値 : [200] 表 6.1.6-2	
3	M0001 が B0001 を借りる。採番リセット済みなので貸出 ID は 1 になる URL : 表 6.1.4-1-[3]	表 6.1.5-1	期待値 : [201] 表 6.1.6-3	
4	応答の貸出 ID を loan_id に保存し、後続の返却 URL へ引き継ぐ変数を設定			
5	DB の LOANS 表に貸出中の行が 1 件あり、返却期限が貸出日+14 日であることを直接確認する 検証のみ		期待値 : 表 6.1.6-4	
6	貸出により B0001 の貸出可能冊数が 1 減ったことを確認する URL : 表 6.1.4-1-[6]		期待値 : [200] 表 6.1.6-5	
7	会員の貸出中一覧に今回の貸出だけが表示されることを確認する URL : 表 6.1.4-1-[7]		期待値 : [200] 表 6.1.6-6	
8	保存した貸出 ID で返却し、状態が RETURNED になり返却日が入ることを確認する URL : 表 6.1.4-1-[8]	表 6.1.5-2	期待値 : [200] 表 6.1.6-7	
9	同じ貸出を再度返却すると 409 ALREADY_RETURNED になることを確認する URL : 表 6.1.4-1-[9]	表 6.1.5-3	期待値 : [409] 表 6.1.6-8	

表 6.1.1-1 テストシナリオ (2/2)

シナリオ	LIB-001 本を借りて返すまでの正常系 (在庫増減・ID 引き継ぎ・二重返却)			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
10	返却により B0001 の貸出可能冊数が初期値に戻り、二重返却 で増えていないことを確認する URL : 表 6.1.4-1-[10]		期待値 : [200] 表 6.1.6-9	
11	会員の貸出中件数が 0 に戻ったことを確認する URL : 表 6.1.4-1-[11]		期待値 : [200] 表 6.1.6-10	

6.1.2 前処理

表 6.1.2-1 前処理 (LIB-001 本を借りて返すまでの正常系 (在庫増減・ID 引き継ぎ・二重返却))

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql

6.1.3 後処理

表 6.1.3-1 後処理 (LIB-001 本を借りて返すまでの正常系 (在庫増減・ID 引き継ぎ・二重返却))

順序	ファイル	出典
1	lib001_assert_after.sql	sql/cases/lib001_assert_after.sql
2	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.1.4 HTTP 呼び出し

表 6.1.4-1 HTTP 呼び出し情報 (LIB-001 本を借りて返すまでの正常系 (在庫増減・ID 引き継ぎ・二重返却)) (1/2)

手順	Method	URL	Headers	Query
1	GET	http://127.0.0.1:18081/members/M0001		
2	GET	http://127.0.0.1:18081/books/B0001		
3	POST	http://127.0.0.1:18081/loans		
6	GET	http://127.0.0.1:18081/books/B0001		

表 6.1.4-1 HTTP 呼び出し情報（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却））（2／2）

手順	Method	URL	Headers	Query
7	GET	http://127.0.0.1:18081/members/M0001/loans? status=ACTIVE		
8	POST	http://127.0.0.1:18081/loans/{ loan_id }/return		
9	POST	http://127.0.0.1:18081/loans/{ loan_id }/return		
10	GET	http://127.0.0.1:18081/books/B0001		
11	GET	http://127.0.0.1:18081/members/M0001		

6.1.5 リクエストボディ

表 6.1.5-1 リクエストボディ（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 3）

JSON path	値
\$.bookId	B0001
\$.memberId	M0001

表 6.1.5-2 リクエストボディ（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 8）

JSON path	値
\$	{}

表 6.1.5-3 リクエストボディ（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 9）

JSON path	値
\$	{}

6.1.6 期待値・検証条件

表 6.1.6-1 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 1）（1／2）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:15	current.res.status == 200 && current.res.body.status == "ACTIVE" && current.res.body.activeLoanCount == 0 && current.res.body.overdue == false

表 6.1.6-1 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 1）（2／2）

区分	出典	内容
ステータス	検証式から抽出	HTTP 200

表 6.1.6-2 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 2）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:26	current.res.status == 200 && current.res.body.totalCopies == 3 && current.res.body.availableCopies == 3
ステータス	検証式から抽出	HTTP 200

表 6.1.6-3 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 3）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:36	current.res.status == 201 && current.res.body.loanId == 1 && current.res.body.memberId == "M0001" && current.res.body.bookId == "B0001" && current.res.body.status == "ACTIVE" && current.res.body.returnedAt == nil && current.res.body.dueDate > current.res.body.loanedAt
ステータス	検証式から抽出	HTTP 201

表 6.1.6-4 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 5）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:57	len(current.rows) == 1 && current.rows[0].STATUS == "ACTIVE" && current.rows[0].DAYS == "14"

表 6.1.6-5 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 6）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:68	current.res.status == 200 && current.res.body.availableCopies == steps.book_before.res.body.available Copies - 1
ステータス	検証式から抽出	HTTP 200

表 6.1.6-6 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 7）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:77	current.res.status == 200 && current.res.body.total == 1 && current.res.body.items[0].loanId == loan_id
ステータス	検証式から抽出	HTTP 200

表 6.1.6-7 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 8）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:87	current.res.status == 200 && current.res.body.loanId == loan_id && current.res.body.status == "RETURNED" && current.res.body.returnedAt != nil
ステータス	検証式から抽出	HTTP 200

表 6.1.6-8 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 9）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:99	current.res.status == 409 && current.res.body.code == "ALREADY_RETURNED"
ステータス	検証式から抽出	HTTP 409

表 6.1.6-9 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 10）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:109	current.res.status == 200 && current.res.body.availableCopies == steps.book_before.res.body.available Copies
ステータス	検証式から抽出	HTTP 200

表 6.1.6-10 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 11）（1／2）

区分	出典	内容
検証式	lib-001-loan-lifecycle .yaml:118	current.res.status == 200 && current.res.body.activeLoanCount == 0

表 6.1.6-10 期待値・検証条件（LIB-001 本を借りて返すまでの正常系（在庫増減・ID 引き継ぎ・二重返却）／手順 11）（2／2）

区分	出典	内容
ステータス	検証式から抽出	HTTP 200

6.2 LIB-002 在庫切れ

- 目的：所蔵 1 冊の本を 2 人が借りようとしたとき、後から借りる会員が 409 NO_STOCK になり、返却後は借りられることを確認する。
- 前提：共通前処理のみ（B0002 は 1 冊）。
- 合格条件：全手順の検証式が真であり、共通後処理（不変条件の検証）が正常終了すること。
- 実行：runnora run --config config.yaml runbooks/scenarios/lib-002-no-stock.yml

6.2.1 テストシナリオ

表 6.2.1-1 テストシナリオ（1／2）

シナリオ	LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	所蔵 1 冊の B0002 を M0001 が借りる URL：表 6.2.4-1-[1]	表 6.2.5-1	期待値：[201] 表 6.2.6-1	
2	1 件目の貸出 ID を first_loan_id に保存する 変数を設定			
3	B0002 の貸出可能冊数が 0 になったことを確認する URL：表 6.2.4-1-[3]		期待値：[200] 表 6.2.6-2	
4	貸出可能な小説の検索結果から B0002 が消えることを確認する URL：表 6.2.4-1-[4]		期待値：[200] 表 6.2.6-3	
5	別会員 M0002 が同じ本を借りようすると 409 NO_STOCK になる URL：表 6.2.4-1-[5]	表 6.2.5-2	期待値：[409] 表 6.2.6-4	
6	拒否された貸出が DB に作られていないことを確認する 検証のみ		期待値：表 6.2.6-5	
7	M0001 が返却する URL：表 6.2.4-1-[7]	表 6.2.5-3	期待値：[200] 表 6.2.6-6	
8	返却後は M0002 が借りられ、新しい貸出 ID が採番される URL：表 6.2.4-1-[8]	表 6.2.5-4	期待値：[201] 表 6.2.6-7	

表 6.2.1-1 テストシナリオ (2/2)

シナリオ	LIB-002 在庫 1 冊の本の取り扱い（在庫切れ 409 と返却後の再貸出）			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
9	再び貸出可能冊数が 0 になったことを確認する URL : 表 6.2.4-1-[9]		期待値 : [200] 表 6.2.6-8	

6.2.2 前処理

表 6.2.2-1 前処理（LIB-002 在庫 1 冊の本の取り扱い（在庫切れ 409 と返却後の再貸出））

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql

6.2.3 後処理

表 6.2.3-1 後処理（LIB-002 在庫 1 冊の本の取り扱い（在庫切れ 409 と返却後の再貸出））

順序	ファイル	出典
1	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.2.4 HTTP 呼び出し

表 6.2.4-1 HTTP 呼び出し情報（LIB-002 在庫 1 冊の本の取り扱い（在庫切れ 409 と返却後の再貸出））

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/loans		
3	GET	http://127.0.0.1:18081/books/B0002		
4	GET	http://127.0.0.1:18081/books?genre=NOVEL&availableOnly=true		
5	POST	http://127.0.0.1:18081/loans		
7	POST	http://127.0.0.1:18081/loans/{ { first_loan_id } }/return		
8	POST	http://127.0.0.1:18081/loans		
9	GET	http://127.0.0.1:18081/books/B0002		

6.2.5 リクエストボディ

表 6.2.5-1 リクエストボディ（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 1）

JSON path	値
\$.bookId	B0002
\$.memberId	M0001

表 6.2.5-2 リクエストボディ（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 5）

JSON path	値
\$.bookId	B0002
\$.memberId	M0002

表 6.2.5-3 リクエストボディ（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 7）

JSON path	値
\$	{ }

表 6.2.5-4 リクエストボディ（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 8）

JSON path	値
\$.bookId	B0002
\$.memberId	M0002

6.2.6 期待値・検証条件

表 6.2.6-1 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 1）

区分	出典	内容
検証式	lib-002-no-stock.yml:14	current.res.status == 201 && current.res.body.loanId == 1 && current.res.body.bookId == "B0002"
ステータス	検証式から抽出	HTTP 201

表 6.2.6-2 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 3）

区分	出典	内容
検証式	lib-002-no-stock.yml:31	current.res.status == 200 && current.res.body.availableCopies == 0
ステータス	検証式から抽出	HTTP 200

表 6.2.6-3 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 4）

区分	出典	内容
検証式	lib-002-no-stock.yml:40	current.res.status == 200 && current.res.body.total == 1 && current.res.body.items[0].bookId == "B0001"
ステータス	検証式から抽出	HTTP 200

表 6.2.6-4 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 5）

区分	出典	内容
検証式	lib-002-no-stock.yml:50	current.res.status == 409 && current.res.body.code == "NO_STOCK"
ステータス	検証式から抽出	HTTP 409

表 6.2.6-5 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 6）

区分	出典	内容
検証式	lib-002-no-stock.yml:62	current.rows[0].CNT == "1"

表 6.2.6-6 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 7）

区分	出典	内容
検証式	lib-002-no-stock.yml:68	current.res.status == 200 && current.res.body.status == "RETURNED"
ステータス	検証式から抽出	HTTP 200

表 6.2.6-7 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 8）

区分	出典	内容
検証式	lib-002-no-stock.yml:78	current.res.status == 201 && current.res.body.loanId == first_loan_id + 1 && current.res.body.memberId == "M0002"
ステータス	検証式から抽出	HTTP 201

表 6.2.6-8 期待値・検証条件（LIB-002 在庫 1 冊の本の取り合い（在庫切れ 409 と返却後の再貸出）／手順 9）

区分	出典	内容
検証式	lib-002-no-stock.yml:91	current.res.status == 200 && current.res.body.availableCopies == 0
ステータス	検証式から抽出	HTTP 200

6.3 LIB-003 延滞中の貸出拒否

- 目的：返却期限を過ぎた貸出がある会員は借りられず（409 OVERDUE_LOAN）、返却すると借りられることを確認する。
- 前提：前処理 lib003_overdue_before.sql が、API では作れない「21 日前に借りて未返却（期限は 7 日前）」の貸出 1 を %ROWTYPE レコードで組み立てて登録し、在庫も 1 減らす。
- 合格条件：全手順の検証式が真であり、共通後処理が正常終了すること。
- 実行：runnora run --config config.yaml --before-sql sql/cases/lib003_overdue_before.sql runbooks/scenarios/lib-003-overdue.yml

6.3.1 テストシナリオ

表 6.3.1-1 テストシナリオ（1／2）

シナリオ	LIB-003 延滞中の会員は借りられず、返却すると借りられる			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	前処理で作った延滞により、M0002 が延滞ありと表示される ことを確認する URL：表 6.3.4-1-[1]		期待値：[200] 表 6.3.6-1	
2	貸出中一覧で、貸出 1（B0004）の返却期限が本日より前であることを確認する URL：表 6.3.4-1-[2]		期待値：[200] 表 6.3.6-2	
3	前処理で在庫も減らしているため、B0004 の貸出可能冊数が 0 であることを確認する URL：表 6.3.4-1-[3]		期待値：[200] 表 6.3.6-3	
4	延滞中の M0002 が別の本 B0001 を借りようすると 409 OVERDUE_LOAN になる URL：表 6.3.4-1-[4]	表 6.3.5-1	期待値：[409] 表 6.3.6-4	
5	延滞している貸出 1 を返却する URL：表 6.3.4-1-[5]	表 6.3.5-2	期待値：[200] 表 6.3.6-5	
6	返却後は延滞なし・貸出 0 件になることを確認する URL：表 6.3.4-1-[6]		期待値：[200] 表 6.3.6-6	

表 6.3.1-1 テストシナリオ (2/2)

シナリオ	LIB-003 延滞中の会員は借りられず、返却すると借りられる			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
7	延滞が解消したので B0001 を借りられる (貸出 ID は 2) URL : 表 6.3.4-1-[7]	表 6.3.5-3	期待値 : [201] 表 6.3.6-7	

6.3.2 前処理

表 6.3.2-1 前処理 (LIB-003 延滞中の会員は借りられず、返却すると借りられる)

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql
3	lib003_overdue_before.sql	sql/cases/lib003_overdue_before.sql

6.3.3 後処理

表 6.3.3-1 後処理 (LIB-003 延滞中の会員は借りられず、返却すると借りられる)

順序	ファイル	出典
1	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.3.4 HTTP 呼び出し

表 6.3.4-1 HTTP 呼び出し情報 (LIB-003 延滞中の会員は借りられず、返却すると借りられる) (1/2)

手順	Method	URL	Headers	Query
1	GET	http://127.0.0.1:18081/members/M0002		
2	GET	http://127.0.0.1:18081/members/M0002/loans? status=ACTI VE		
3	GET	http://127.0.0.1:18081/books/B0004		
4	POST	http://127.0.0.1:18081/loans		
5	POST	http://127.0.0.1:18081/loans/1/return		
6	GET	http://127.0.0.1:18081/members/M0002		

表 6.3.4-1 HTTP 呼び出し情報 (LIB-003 延滞中の会員は借りられず、返却すると借りられる) (2/2)

手順	Method	URL	Headers	Query
7	POST	http://127.0.0.1:18081/loans		

6.3.5 リクエストボディ

表 6.3.5-1 リクエストボディ (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 4)

JSON path	値
\$.bookId	B0001
\$.memberId	M0002

表 6.3.5-2 リクエストボディ (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 5)

JSON path	値
\$	{}

表 6.3.5-3 リクエストボディ (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 7)

JSON path	値
\$.bookId	B0001
\$.memberId	M0002

6.3.6 期待値・検証条件

表 6.3.6-1 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 1)

区分	出典	内容
検証式	lib-003-overdue.yml:16	current.res.status == 200 && current.res.body.activeLoanCount == 1 && current.res.body.overdue == true
ステータス	検証式から抽出	HTTP 200

表 6.3.6-2 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 2)

区分	出典	内容
検証式	lib-003-overdue.yml:26	current.res.status == 200 && current.res.body.total == 1 && current.res.body.items[0].loanId == 1 && current.res.body.items[0].bookId == "B0004" && date(current.res.body.items[0].dueDate) < now()
ステータス	検証式から抽出	HTTP 200

表 6.3.6-3 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 3)

区分	出典	内容
検証式	lib-003-overdue.yml:38	current.res.status == 200 && current.res.body.availableCopies == 0
ステータス	検証式から抽出	HTTP 200

表 6.3.6-4 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 4)

区分	出典	内容
検証式	lib-003-overdue.yml:47	current.res.status == 409 && current.res.body.code == "OVERDUE_LOAN"
ステータス	検証式から抽出	HTTP 409

表 6.3.6-5 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 5)

区分	出典	内容
検証式	lib-003-overdue.yml:59	current.res.status == 200 && current.res.body.status == "RETURNED"
ステータス	検証式から抽出	HTTP 200

表 6.3.6-6 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 6)

区分	出典	内容
検証式	lib-003-overdue.yml:69	current.res.status == 200 && current.res.body.activeLoanCount == 0 && current.res.body.overdue == false
ステータス	検証式から抽出	HTTP 200

表 6.3.6-7 期待値・検証条件 (LIB-003 延滞中の会員は借りられず、返却すると借りられる／手順 7)

区分	出典	内容
検証式	lib-003-overdue.yml:79	current.res.status == 201 && current.res.body.loanId == 2
ステータス	検証式から抽出	HTTP 201

6.4 LIB-004 貸出上限

- 目的：上限（3 冊）まで借りている会員は 4 冊目を借りられず（409 LOAN_LIMIT_EXCEEDED）、1 冊返すと借りられることを確認する。
- 前提：前処理 `lib004_fill_loans_before.sql` が、パラメータ付き明示カーソルと WHILE LOOP で 貸出可能な TECH の本を `book_id` 順に上限まで貸し出す（貸出 1～3 = B0003, B0006, B0008）。
- 合格条件：全手順の検証式が真であり、共通後処理が正常終了すること。
- 実行：`runnora run --config config.yaml --before-sql sql/cases/lib004_fill_loans_before.sql runbooks/scenarios/lib-004-loan-limit.yml`

6.4.1 テストシナリオ

表 6.4.1-1 テストシナリオ

シナリオ	LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	前処理により M0001 の貸出中件数が上限 3 に達していることを 確認する URL：表 6.4.4-1-[1]		期待値：[200] 表 6.4.6-1	
2	明示カーソルが <code>book_id</code> 順に貸し出した 3 冊を確認する URL：表 6.4.4-1-[2]		期待値：[200] 表 6.4.6-2	
3	4 冊目を借りようすると 409 LOAN_LIMIT_EXCEEDED になる URL：表 6.4.4-1-[3]	表 6.4.5-1	期待値：[409] 表 6.4.6-3	
4	貸出 2（B0006）を返却する URL：表 6.4.4-1-[4]	表 6.4.5-2	期待値：[200] 表 6.4.6-4	
5	上限に空きができたので B0001 を借りられる（貸出 ID は 4） URL：表 6.4.4-1-[5]	表 6.4.5-3	期待値：[201] 表 6.4.6-5	
6	DB 上でも M0001 の貸出中が 3 件（上限ちょうど）であることを 確認する 検証のみ		期待値：表 6.4.6-6	

6.4.2 前処理

表 6.4.2-1 前処理（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる）

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql
3	lib004_fill_loans_before.sql	sql/cases/lib004_fill_loans_before.sql

6.4.3 後処理

表 6.4.3-1 後処理（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる）

順序	ファイル	出典
1	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.4.4 HTTP 呼び出し

表 6.4.4-1 HTTP 呼び出し情報（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる）

手順	Method	URL	Headers	Query
1	GET	http://127.0.0.1:18081/members/M0001		
2	GET	http://127.0.0.1:18081/members/M0001/loans? status=ACTI VE		
3	POST	http://127.0.0.1:18081/loans		
4	POST	http://127.0.0.1:18081/loans/2/return		
5	POST	http://127.0.0.1:18081/loans		

6.4.5 リクエストボディ

表 6.4.5-1 リクエストボディ（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 3）

JSON path	値
\$.bookId	B0001
\$.memberId	M0001

表 6.4.5-2 リクエストボディ (LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 4)

JSON path	値
\$	{}

表 6.4.5-3 リクエストボディ (LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 5)

JSON path	値
\$.bookId	B0001
\$.memberId	M0001

6.4.6 期待値・検証条件

表 6.4.6-1 期待値・検証条件 (LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 1)

区分	出典	内容
検証式	lib-004-loan-limit .yaml:17	current.res.status == 200 && current.res.body.activeLoanCount == 3 && current.res.body.maxLoans == 3
ステータス	検証式から抽出	HTTP 200

表 6.4.6-2 期待値・検証条件 (LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 2)

区分	出典	内容
検証式	lib-004-loan-limit .yaml:27	current.res.status == 200 && current.res.body.total == 3 && map(current.res.body.items, .bookId) == ["B0003", "B0006", "B0008"] && map(current.res.body.items, .loanId) == [1, 2, 3]
ステータス	検証式から抽出	HTTP 200

表 6.4.6-3 期待値・検証条件 (LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 3)

区分	出典	内容
検証式	lib-004-loan-limit .yaml:38	current.res.status == 409 && current.res.body.code == "LOAN_LIMIT_EXCEEDED"
ステータス	検証式から抽出	HTTP 409

表 6.4.6-4 期待値・検証条件（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 4）

区分	出典	内容
検証式	lib-004-loan-limit .yaml:50	current.res.status == 200 && current.res.body.bookId == "B0006"
ステータス	検証式から抽出	HTTP 200

表 6.4.6-5 期待値・検証条件（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 5）

区分	出典	内容
検証式	lib-004-loan-limit .yaml:60	current.res.status == 201 && current.res.body loanId == 4
ステータス	検証式から抽出	HTTP 201

表 6.4.6-6 期待値・検証条件（LIB-004 貸出上限 3 冊に達した会員は借りられず、1 冊返すと借りられる／手順 6）

区分	出典	内容
検証式	lib-004-loan-limit .yaml:72	current.rows[0].CNT == "3"

6.5 LIB-005 SAVEPOINT による部分確定

- 目的：前処理で SAVEPOINT / ROLLBACK TO SAVEPOINT を使い、失敗した単位だけを取り消した初期データが API から期待どおりに見えることを確認する。
- 前提：前処理 `lib005_savepoint_before.sql` が「蔵書登録 + 初回貸出」を 1 単位として 5 件投入する。

単位	蔵書	内容	結果
1	B0101	2 冊、M0004 に初回貸出	確定（貸出 1）
2	B0102	1 冊	確定
3	B0103	B0101 と同じ ISBN	DUP_VAL_ON_INDEX → ROLLBACK TO SAVEPOINT
4	B0104	存在しない会員 M9999 に初回貸出	蔵書は登録済みだが貸出で ORA-02291 → 蔵書ごと取消（貸出 2 は欠番）
5	B0105	3 冊、M0005 に初回貸出	確定（貸出 3）

後半で M0006～M0008 を一括停止し、件数超過として ROLLBACK TO SAVEPOINT で取り消したうえで M0008 だけを停止する。

- 合格条件：全手順の検証式が真であり、後処理 `lib005_assert_after.sql` が「TEST 蔵書 4 件（前処理 3 + API 登録 1）・B0104 なし・ROLLBACK のログ 2 件以上」を確認して正常終了すること。
- 実行：`runnora run --config config.yaml --before-sql sql/cases/lib005_savepoint_before.sql --after-sql sql/cases/lib005_assert_after.sql runbooks/scenarios/lib-005-savepoint.yml`

6.5.1 テストシナリオ

表 6.5.1-1 テストシナリオ（1／2）

シナリオ	LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	TEST ジャンルの蔵書が確定した 3 件だけであることを確認する URL：表 6.5.4-1-[1]		期待値：[200] 表 6.5.6-1	
2	ISBN 重複で取り消した B0103 が存在しないことを確認する URL：表 6.5.4-1-[2]		期待値：[404] 表 6.5.6-2	

表 6.5.1-1 テストシナリオ (2/2)

シナリオ	LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
3	貸出の外部キー違反で蔵書ごと取り消した B0104 が存在しないことを確認する URL : 表 6.5.4-1-[3]		期待値 : [404] 表 6.5.6-3	
4	蔵書と初回貸出を同じ単位で確定した B0101 は、貸出可能冊数が 1 減っている URL : 表 6.5.4-1-[4]		期待値 : [200] 表 6.5.6-4	
5	取り消した単位で消費された採番は戻らないため、B0105 の 貸出 ID は 3 になる URL : 表 6.5.4-1-[5]		期待値 : [200] 表 6.5.6-5	
6	一括停止は取り消されたため、M0006 は利用可能のまま URL : 表 6.5.4-1-[6]		期待値 : [200] 表 6.5.6-6	
7	個別停止に切り替えた M0008 だけが SUSPENDED になっている URL : 表 6.5.4-1-[7]		期待値 : [200] 表 6.5.6-7	
8	停止中の M0008 は借りられない (409 MEMBER_SUSPENDED) URL : 表 6.5.4-1-[8]	表 6.5.5-1	期待値 : [409] 表 6.5.6-8	
9	API からも B0101 と同じ ISBN では登録できない (409 DUPLICATE_BOOK) URL : 表 6.5.4-1-[9]	表 6.5.5-2	期待値 : [409] 表 6.5.6-9	
10	正しい ISBN で B0103 を登録し直すと 201 で登録される URL : 表 6.5.4-1-[10]	表 6.5.5-3	期待値 : [201] 表 6.5.6-10	
11	自律型トランザクションで書いた ROLLBACK のログが残っていることを DB で確認する 検証のみ		期待値 : 表 6.5.6-11	

6.5.2 前処理

表 6.5.2-1 前処理（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する）

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql
3	lib005_savepoint_before.sql	sql/cases/lib005_savepoint_before.sql

6.5.3 後処理

表 6.5.3-1 後処理（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する）

順序	ファイル	出典
1	lib005_assert_after.sql	sql/cases/lib005_assert_after.sql
2	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.5.4 HTTP 呼び出し

表 6.5.4-1 HTTP 呼び出し情報（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する）

手順	Method	URL	Headers	Query
1	GET	http://127.0.0.1:18081/books?genre=TEST		
2	GET	http://127.0.0.1:18081/books/B0103		
3	GET	http://127.0.0.1:18081/books/B0104		
4	GET	http://127.0.0.1:18081/books/B0101		
5	GET	http://127.0.0.1:18081/members/M0005/loans		
6	GET	http://127.0.0.1:18081/members/M0006		
7	GET	http://127.0.0.1:18081/members/M0008		
8	POST	http://127.0.0.1:18081/loans		
9	POST	http://127.0.0.1:18081/books		
10	POST	http://127.0.0.1:18081/books		

6.5.5 リクエストボディ

表 6.5.5-1 リクエストボディ（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 8）

JSON path	値
\$.bookId	B0102
\$.memberId	M0008

表 6.5.5-2 リクエストボディ（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 9）

JSON path	値
\$.author	試験 花子
\$.bookId	B0103
\$.genre	TEST
\$.isbn	9784000000101
\$.title	再登録テスト
\$.totalCopies	1

表 6.5.5-3 リクエストボディ（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 10）

JSON path	値
\$.author	試験 花子
\$.bookId	B0103
\$.genre	TEST
\$.isbn	9784000000103
\$.title	再登録テスト
\$.totalCopies	1

6.5.6 期待値・検証条件

表 6.5.6-1 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 1）

区分	出典	内容
検証式	lib-005-savepoint.yml:20	current.res.status == 200 && current.res.body.total == 3 && map(current.res.body.items, .bookId) == ["B0101", "B0102", "B0105"]
ステータス	検証式から抽出	HTTP 200

表 6.5.6-2 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 2）

区分	出典	内容
検証式	lib-005-savepoint.yml:30	current.res.status == 404 && current.res.body.code == "NOT_FOUND"
ステータス	検証式から抽出	HTTP 404

表 6.5.6-3 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 3）

区分	出典	内容
検証式	lib-005-savepoint.yml:39	current.res.status == 404
ステータス	検証式から抽出	HTTP 404

表 6.5.6-4 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 4）

区分	出典	内容
検証式	lib-005-savepoint.yml:46	current.res.status == 200 && current.res.body.totalCopies == 2 && current.res.body.availableCopies == 1
ステータス	検証式から抽出	HTTP 200

表 6.5.6-5 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 5）（1／2）

区分	出典	内容
検証式	lib-005-savepoint.yml:56	current.res.status == 200 && current.res.body.total == 1 && current.res.body.items[0].loanId == 3 && current.res.body.items[0].bookId == "B0105"

表 6.5.6-5 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 5）（2／2）

区分	出典	内容
ステータス	検証式から抽出	HTTP 200

表 6.5.6-6 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 6）

区分	出典	内容
検証式	lib-005-savepoint.yml:67	current.res.status == 200 && current.res.body.status == "ACTIVE"
ステータス	検証式から抽出	HTTP 200

表 6.5.6-7 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 7）

区分	出典	内容
検証式	lib-005-savepoint.yml:76	current.res.status == 200 && current.res.body.status == "SUSPENDED"
ステータス	検証式から抽出	HTTP 200

表 6.5.6-8 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 8）

区分	出典	内容
検証式	lib-005-savepoint.yml:85	current.res.status == 409 && current.res.body.code == "MEMBER_SUSPENDED"
ステータス	検証式から抽出	HTTP 409

表 6.5.6-9 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 9）

区分	出典	内容
検証式	lib-005-savepoint.yml:97	current.res.status == 409 && current.res.body.code == "DUPLICATE_BOOK"
ステータス	検証式から抽出	HTTP 409

表 6.5.6-10 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 10）（1／2）

区分	出典	内容
検証式	lib-005-savepoint .yml:113	current.res.status == 201 && current.res.body.bookId == "B0103" && current.res.body.availableCopies == 1

表 6.5.6-10 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 10）（2／2）

区分	出典	内容
ステータス	検証式から抽出	HTTP 201

表 6.5.6-11 期待値・検証条件（LIB-005 SAVEPOINT で一部だけ確定した初期データを API から確認する／手順 11）

区分	出典	内容
検証式	lib-005-savepoint .yaml:130	int(current.rows[0].CNT) >= 3

6.6 LIB-006 異常系

- 目的：入力不正（400）・対象なし（404）・利用停止（409）の各エラーが正しく返り、DB を変更しないことを確認する。
- 前提：共通前処理のみ（M0003 は利用停止）。
- 合格条件：全手順の検証式が真であり、後処理 `lib006_assert_no_loans_after.sql` が「貸出 0 件・蔵書 20 件・貸出 ID の採番が進んでいない」を確認して正常終了すること。
- 実行：`runnora run --config config.yaml --after-sql sql/cases/lib006_assert_no_loans_after.sql runbooks/scenarios/lib-006-validation-errors.yml`

6.6.1 テストシナリオ

表 6.6.1-1 テストシナリオ（1／2）

シナリオ	LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない			
手順番号	試験手順		期待結果	
	手順内容	リクエストボディ	ステータス・期待値	結果
1	利用停止中の会員 M0003 は借りられない URL：表 6.6.4-1-[1]	表 6.6.5-1	期待値：[409] 表 6.6.6-1	
2	存在しない会員 M9999 は 404 URL：表 6.6.4-1-[2]	表 6.6.5-2	期待値：[404] 表 6.6.6-2	
3	存在しない蔵書 B9999 は 404 URL：表 6.6.4-1-[3]	表 6.6.5-3	期待値：[404] 表 6.6.6-3	
4	bookId が無いリクエストは 400 URL：表 6.6.4-1-[4]	表 6.6.5-4	期待値：[400] 表 6.6.6-4	
5	定義に無い項目を含むリクエストは 400 URL：表 6.6.4-1-[5]	表 6.6.5-5	期待値：[400] 表 6.6.6-5	
6	13 桁でない ISBN では蔵書を登録できない URL：表 6.6.4-1-[6]	表 6.6.5-6	期待値：[400] 表 6.6.6-6	
7	所蔵冊数 0 では蔵書を登録できない URL：表 6.6.4-1-[7]	表 6.6.5-7	期待値：[400] 表 6.6.6-7	
8	定義外のジャンルでの検索は 400 URL：表 6.6.4-1-[8]		期待値：[400] 表 6.6.6-8	

表 6.6.1-1 テストシナリオ (2/2)

シナリオ	LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
9	定義外の貸出状態での絞り込みは 400 URL : 表 6.6.4-1-[9]		期待値 : [400] 表 6.6.6-9	
10	存在しない貸出の返却は 404 URL : 表 6.6.4-1-[10]	表 6.6.5-8	期待値 : [404] 表 6.6.6-10	

6.6.2 前処理

表 6.6.2-1 前処理 (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない)

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql

6.6.3 後処理

表 6.6.3-1 後処理 (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない)

順序	ファイル	出典
1	lib006_assert_no_loans_after.s ql	sql/cases/lib006_assert_no_loans_after.sql
2	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.6.4 HTTP 呼び出し

表 6.6.4-1 HTTP 呼び出し情報 (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない) (1/2)

手順	Method	URL	Headers	Query
1	POST	http://127.0.0.1:18081/loans		
2	POST	http://127.0.0.1:18081/loans		
3	POST	http://127.0.0.1:18081/loans		
4	POST	http://127.0.0.1:18081/loans		
5	POST	http://127.0.0.1:18081/loans		

表 6.6.4-1 HTTP 呼び出し情報 (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない) (2/2)

手順	Method	URL	Headers	Query
6	POST	http://127.0.0.1:18081/books		
7	POST	http://127.0.0.1:18081/books		
8	GET	http://127.0.0.1:18081/books?genre=COMIC		
9	GET	http://127.0.0.1:18081/members/M0001/loans? status=LOST		
10	POST	http://127.0.0.1:18081/loans/9999/return		

6.6.5 リクエストボディ

表 6.6.5-1 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない/手順 1)

JSON path	値
\$.bookId	B0001
\$.memberId	M0003

表 6.6.5-2 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない/手順 2)

JSON path	値
\$.bookId	B0001
\$.memberId	M9999

表 6.6.5-3 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない/手順 3)

JSON path	値
\$.bookId	B9999
\$.memberId	M0001

表 6.6.5-4 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない/手順 4)

JSON path	値
\$.memberId	M0001

表 6.6.5-5 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない／手順 5)

JSON path	値
\$.bookId	B0001
\$.days	30
\$.memberId	M0001

表 6.6.5-6 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない／手順 6)

JSON path	値
\$.author	試験 花子
\$.bookId	B0901
\$.genre	TECH
\$.isbn	978400
\$.title	不正 ISBN
\$.totalCopies	1

表 6.6.5-7 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない／手順 7)

JSON path	値
\$.author	試験 花子
\$.bookId	B0902
\$.genre	TECH
\$.isbn	9784000000902
\$.title	冊数不正
\$.totalCopies	0

表 6.6.5-8 リクエストボディ (LIB-006 異常系 (入力不正 400・存在しない 404・利用停止 409) で DB が変わらない／手順 10)

JSON path	値
\$	{}

6.6.6 期待値・検証条件

表 6.6.6-1 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 1）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:14	current.res.status == 409 && current.res.body.code == "MEMBER_SUSPENDED"
ステータス	検証式から抽出	HTTP 409

表 6.6.6-2 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 2）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:26	current.res.status == 404 && current.res.body.code == "NOT_FOUND"
ステータス	検証式から抽出	HTTP 404

表 6.6.6-3 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 3）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:38	current.res.status == 404 && current.res.body.code == "NOT_FOUND"
ステータス	検証式から抽出	HTTP 404

表 6.6.6-4 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 4）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:50	current.res.status == 400 && current.res.body.code == "VALIDATION_ERROR" && current.res.body.message == "bookId is required"
ステータス	検証式から抽出	HTTP 400

表 6.6.6-5 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 5）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:62	current.res.status == 400 && current.res.body.code == "VALIDATION_ERROR"
ステータス	検証式から抽出	HTTP 400

表 6.6.6-6 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 6）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:75	current.res.status == 400 && current.res.body.message == "isbn must be 13 digits"
ステータス	検証式から抽出	HTTP 400

表 6.6.6-7 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 7）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:91	current.res.status == 400 && current.res.body.message == "totalCopies must be between 1 and 99"
ステータス	検証式から抽出	HTTP 400

表 6.6.6-8 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 8）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:107	current.res.status == 400 && current.res.body.code == "VALIDATION_ERROR"
ステータス	検証式から抽出	HTTP 400

表 6.6.6-9 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 9）（1／2）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:116	current.res.status == 400

表 6.6.6-9 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 9）（2／2）

区分	出典	内容
ステータス	検証式から抽出	HTTP 400

表 6.6.6-10 期待値・検証条件（LIB-006 異常系（入力不正 400・存在しない 404・利用停止 409）で DB が変わらない／手順 10）

区分	出典	内容
検証式	lib-006-validation-errors .yaml:123	current.res.status == 404 && current.res.body.code == "NOT_FOUND"
ステータス	検証式から抽出	HTTP 404

6.7 LIB-007 大量の貸出履歴

- 目的：前処理で一括投入した大量の返却済み履歴が正しく照会でき、延滞や上限の判定に影響しないことを確認する。
- 前提：前処理 `lib007_history_before.sql` が BULK COLLECT ... LIMIT 5 と FORALL で M0004 に B0005～B0020 の返却済み貸出 16 件（貸出 1～16）を 4 バッチで投入する。
- 合格条件：全手順の検証式が真であり、共通後処理が正常終了すること。
- 実行：`runnora run --config config.yaml --before-sql sql/cases/lib007_history_before.sql runbooks/scenarios/lib-007-bulk-history.yml`

6.7.1 テストシナリオ

表 6.7.1-1 テストシナリオ

シナリオ	LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する			
手順番号	試験手順	期待結果		結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	M0004 の返却済み履歴が 16 件、貸出 ID 順に並んでいることを確認する URL：表 6.7.4-1-[1]		期待値：[200] 表 6.7.6-1	
2	貸出中の履歴は 0 件である URL：表 6.7.4-1-[2]		期待値：[200] 表 6.7.6-2	
3	返却済みの貸出は期限を過ぎていても延滞にならない URL：表 6.7.4-1-[3]		期待値：[200] 表 6.7.6-3	
4	履歴が多くても新規貸出でき、貸出 ID は 17 になる URL：表 6.7.4-1-[4]	表 6.7.5-1	期待値：[201] 表 6.7.6-4	
5	前処理が BULK COLLECT LIMIT 5 で 4 バッチに分けて投入したことをログで確認する 検証のみ		期待値：表 6.7.6-5	

6.7.2 前処理

表 6.7.2-1 前処理（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する）（1／2）

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql

表 6.7.2-1 前処理（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する）（2／2）

順序	ファイル	出典
2	10_seed_master.sql	sql/common/10_seed_master.sql
3	lib007_history_before.sql	sql/cases/lib007_history_before.sql

6.7.3 後処理

表 6.7.3-1 後処理（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する）

順序	ファイル	出典
1	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.7.4 HTTP 呼び出し

表 6.7.4-1 HTTP 呼び出し情報（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する）

手順	Method	URL	Headers	Query
1	GET	http://127.0.0.1:18081/members/M0004/loans? status=RETURNED		
2	GET	http://127.0.0.1:18081/members/M0004/loans? status=ACTIVE		
3	GET	http://127.0.0.1:18081/members/M0004		
4	POST	http://127.0.0.1:18081/loans		

6.7.5 リクエストボディ

表 6.7.5-1 リクエストボディ（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する／手順 4）

JSON path	値
\$.bookId	B0005
\$.memberId	M0004

6.7.6 期待値・検証条件

表 6.7.6-1 期待値・検証条件（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する／手順 1）

区分	出典	内容
検証式	lib-007-bulk-history .yaml:17	current.res.status == 200 && current.res.body.total == 16 && current.res.body.items[0].loanId == 1 && current.res.body.items[0].bookId == "B0005" && current.res.body.items[15].loanId == 16 && current.res.body.items[15].bookId == "B0020" && all(current.res.body.items, .returnedAt != nil)
ステータス	検証式から抽出	HTTP 200

表 6.7.6-2 期待値・検証条件（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する／手順 2）

区分	出典	内容
検証式	lib-007-bulk-history .yaml:31	current.res.status == 200 && current.res.body.total == 0
ステータス	検証式から抽出	HTTP 200

表 6.7.6-3 期待値・検証条件（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する／手順 3）

区分	出典	内容
検証式	lib-007-bulk-history .yaml:40	current.res.status == 200 && current.res.body.activeLoanCount == 0 && current.res.body.overdue == false
ステータス	検証式から抽出	HTTP 200

表 6.7.6-4 期待値・検証条件（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する／手順 4）

区分	出典	内容
検証式	lib-007-bulk-history .yaml:50	current.res.status == 201 && current.res.body.loanId == 17
ステータス	検証式から抽出	HTTP 201

表 6.7.6-5 期待値・検証条件（LIB-007 BULK COLLECT/FORALL で投入した大量の貸出履歴を照会する／手順 5）

区分	出典	内容
検証式	lib-007-bulk-history .yaml:62	current.rows[0].MESSAGE == "batches=4, rows=16"

6.8 DEMO 事後検証による不整合の検知

- 目的：共通後処理 `90_verify_integrity.sql` が DB の不整合を実際に検知できることを確認する（検証の検証）。
- 前提：runbook 自体は稼働確認だけを行い成功する。ケース後処理 `demo_break_invariant_after.sql` が B0001 の貸出可能冊数をわざと 1 減らす。ケース後処理は共通後処理より先に実行される。
- 合格条件：runnora が**終了コード 4** で終了し、エラーに `ORA-20100: integrity check failed (1): B0001 available=2 expected=3` が含まれること。
- 実行：`runnora run --config config.yaml --after-sql sql/cases/demo_break_invariant_after.sql runbooks/demo/hook-failure-detection.yml`

6.8.1 テストシナリオ

表 6.8.1-1 テストシナリオ

シナリオ	DEMO 事後処理の整合性検証が DB 不整合を検知できることの確認（exit 4 が期待値）			
手順番号	試験手順		期待結果	結果
	手順内容	リクエストボディ	ステータス・期待値	良否
1	API と DB が稼働していることを確認する URL：表 6.8.4-1-[1]		期待値：[200] 表 6.8.5-1	

6.8.2 前処理

表 6.8.2-1 前処理（DEMO 事後処理の整合性検証が DB 不整合を検知できることの確認（exit 4 が期待値））

順序	ファイル	出典
1	00_reset.sql	sql/common/00_reset.sql
2	10_seed_master.sql	sql/common/10_seed_master.sql

6.8.3 後処理

表 6.8.3-1 後処理（DEMO 事後処理の整合性検証が DB 不整合を検知できることの確認（exit 4 が期待値））

順序	ファイル	出典
1	demo_break_invariant_after.sql	sql/cases/demo_break_invariant_after.sql
2	90_verify_integrity.sql	sql/common/90_verify_integrity.sql

6.8.4 HTTP 呼び出し

表 6.8.4-1 HTTP 呼び出し情報（DEMO 事後処理の整合性検証が DB 不整合を検知できることの確認（exit 4 が期待値））

手順	Method	URL	Headers	Query
1	GET	http://127.0.0.1:18081/health		

6.8.5 期待値・検証条件

表 6.8.5-1 期待値・検証条件（DEMO 事後処理の整合性検証が DB 不整合を検知できることの確認（exit 4 が期待値）／手順 1）

区分	出典	内容
検証式	hook-failure-detection .yaml:15	current.res.status == 200 && current.res.body.status == "UP" && current.res.body.database == "UP"
ステータス	検証式から抽出	HTTP 200

7. 実施記録

実施者が記入する。証跡は reports/<日時>-mock/、reports/<日時>-api/ に保存する。

表 7-1 実施記録

区分	対象	期待	結果	実施日	実施者	証跡
生成テスト	モック (8 suite)	終了コード 0				mock-generated.txt
契約テスト	モック (8 suite)	終了コード 0				mock-contract.txt
生成テスト	実 API (8 suite)	終了コード 0				api-generated.txt
契約テスト	実 API (8 suite)	終了コード 0				api-contract.txt
シナリオ	LIB-001	終了コード 0				lib-001.txt
シナリオ	LIB-002	終了コード 0				lib-002.txt
シナリオ	LIB-003	終了コード 0				lib-003.txt
シナリオ	LIB-004	終了コード 0				lib-004.txt
シナリオ	LIB-005	終了コード 0				lib-005.txt
シナリオ	LIB-006	終了コード 0				lib-006.txt
シナリオ	LIB-007	終了コード 0				lib-007.txt
検知確認	DEMO	終了コード 4				demo-hook-failure.log
カバレッジ	契約 + シナリオ	8 / 8 operation				coverage.txt

不具合・所見

表 7-2 不具合・所見

No.	内容	対応

8. 付録 前後処理 SQL 全文

第 5・6 章の前処理・後処理の表 (runnora-docgen 生成) は実行順とファイル名だけを示す。表の「出典」に挙げた SQL/PL/SQL ファイルの全文をここに掲載する (生成)。構文と用途の対応は 表 3.3.3-1 を参照。

8.1 sql/common/

8.1.1 00_reset.sql

```
-- =====
-- common/00_reset.sql (config.yaml hooks.common.before 1/2)
-- 全テーブルのデータを削除し、貸出 ID の採番を 1 に戻す。
-- 毎回同じ初期状態から runbook を始めるための「リセット」フック。
--
-- 使用構文: VARRAY / 数値 FOR LOOP / EXECUTE IMMEDIATE (DML・DDL)
--          SQL%ROWCOUNT / DBMS_ASSERT / 局所プロシージャ
--          PRAGMA AUTONOMOUS_TRANSACTION
-- 注意: runnora はファイル全体を 1 文で実行するため、末尾に "/" を書かない。
-- =====

DECLARE
  TYPE t_table_names IS VARRAY(10) OF VARCHAR2(30);
  -- 外部キーがあるため 子 → 親 の順に削除する
  c_tables CONSTANT t_table_names := t_table_names('LOANS', 'BOOKS', 'MEMBERS');
  v_summary VARCHAR2(4000);

  -- 本体のトランザクションとは独立にログを確定させる (ROLLBACK されても残る)
  PROCEDURE log_hook(p_message IN VARCHAR2) IS
    PRAGMA AUTONOMOUS_TRANSACTION;
  BEGIN
    INSERT INTO test_hook_log (hook_name, message) VALUES ('common/00_reset', p_message);
    COMMIT;
  END log_hook;

BEGIN
  FOR i IN 1 .. c_tables.COUNT LOOP
    -- 表名を動的 SQL に連結するので DBMS_ASSERT で識別子として検証する
    EXECUTE IMMEDIATE 'DELETE FROM ' || DBMS_ASSERT.SIMPLE_SQL_NAME(c_tables(i));
    v_summary := v_summary || c_tables(i) || '=' || SQL%ROWCOUNT || ' ';
  END LOOP;

  -- 1 日より古いフックログを掃除する
  DELETE FROM test_hook_log WHERE logged_at < SYSTIMESTAMP - INTERVAL '1' DAY;
  COMMIT;

  -- DDL は暗黙コミットされるため、DML を確定させた後に実行する
  EXECUTE IMMEDIATE 'ALTER SEQUENCE loan_seq RESTART START WITH 1';
```

```
log_hook('deleted rows: ' || RTRIM(v_summary) || ', loan_seq restarted');
END;
```

8.1.2 10_seed_master.sql

```
-- =====
-- common/10_seed_master.sql (config.yaml hooks.common.before 2/2)
-- 会員・蔵書のマスタデータを投入する。
-- OpenAPI の example (B0001, M0001 など) と同じ値にしてあるため、
-- 生成テスト (runnora generate) とモック (oapi2wire) の期待値が実 API でも成り立つ。
--
-- 使用構文: MERGE / レコード型 + 連想配列 (INDEX BY) / 修飾式 (18c+)
--          数値 FOR LOOP / CASE 式 / 局所ファンクション
-- =====

DECLARE
  TYPE t_book IS RECORD (
    book_id books.book_id%TYPE,
    title   books.title%TYPE,
    author  books.author%TYPE,
    genre   books.genre%TYPE,
    copies  books.total_copies%TYPE
  );
  TYPE t_book_tab IS TABLE OF t_book INDEX BY PLS_INTEGER;
  v_books  t_book_tab;
  v_genre  books.genre%TYPE;
  v_copies books.total_copies%TYPE;
  v_isbn   books.isbn%TYPE;

  -- ISBN は 978400000 + 4 桁連番 (B0001 → 9784000000001)
  -- 局所ファンクションは SQL 文の中から呼べない (PLS-00231) ので、変数に代入してから使う
  FUNCTION isbn_of(p_no IN PLS_INTEGER) RETURN VARCHAR2 IS
  BEGIN
    RETURN '978400000' || TO_CHAR(p_no, 'FM0000');
  END isbn_of;

BEGIN
  -- 1) 名前付きの会員 : MERGE で「あれば更新・なければ登録」
  MERGE INTO members m
  USING (
    SELECT 'M0001' AS member_id, '山田 太郎' AS name, 'ACTIVE' AS status FROM dual UNION ALL
    SELECT 'M0002', '佐藤 花子', 'ACTIVE' FROM dual UNION ALL
    SELECT 'M0003', '鈴木 一郎', 'SUSPENDED' FROM dual
  ) s
  ON (m.member_id = s.member_id)
  WHEN MATCHED THEN
```

```

UPDATE SET m.name = s.name, m.status = s.status, m.max_loans = 3
WHEN NOT MATCHED THEN
  INSERT (member_id, name, status, max_loans) VALUES (s.member_id, s.name, s.status, 3);

-- 2) 汎用テスト会員 M0004 ~ M0010 : 数値 FOR LOOP
FOR i IN 4 .. 10 LOOP
  INSERT INTO members (member_id, name, status, max_loans)
  VALUES ('M' || TO_CHAR(i, 'FM0000'), 'テスト会員' || TO_CHAR(i, 'FM00'), 'ACTIVE', 3);
END LOOP;

-- 3) 名前付きの蔵書 : レコードの連想配列を修飾式で組み立てて FOR LOOP で登録
v_books(1) := t_book('B0001', '吾輩は猫である', '夏目漱石', 'NOVEL', 3);
v_books(2) := t_book('B0002', '羅生門', '芥川龍之介', 'NOVEL', 1);
v_books(3) := t_book('B0003', 'Go 言語による Web アプリ開発', '技術 一郎', 'TECH', 2);
v_books(4) := t_book('B0004', '日本の城郭史', '歴史 次郎', 'HISTORY', 1);

FOR i IN 1 .. v_books.COUNT LOOP
  v_isbn := isbn_of(i);
  INSERT INTO books (book_id, isbn, title, author, genre, total_copies, available_copies)
  VALUES (v_books(i).book_id, v_isbn, v_books(i).title, v_books(i).author,
    v_books(i).genre, v_books(i).copies, v_books(i).copies);
END LOOP;

-- 4) 汎用テスト図書 B0005 ~ B0020 : ジャンルと冊数を CASE 式で振り分ける
-- (NOVEL は B0001/B0002 の 2 冊だけにして、OpenAPI example の件数と一致させる)
FOR i IN 5 .. 20 LOOP
  v_genre := CASE MOD(i, 2) WHEN 0 THEN 'TECH' ELSE 'HISTORY' END;
  v_copies := CASE WHEN i <= 10 THEN 1 ELSE 2 END;
  v_isbn := isbn_of(i);
  INSERT INTO books (book_id, isbn, title, author, genre, total_copies, available_copies)
  VALUES ('B' || TO_CHAR(i, 'FM0000'), v_isbn, 'テスト図書' || TO_CHAR(i, 'FM00'),
    'テスト 著者', v_genre, v_copies, v_copies);
END LOOP;

COMMIT;
END;

```

8.1.3 90_verify_integrity.sql

```

-- =====
-- common/90_verify_integrity.sql (config.yaml hooks.common.after)
-- runbook 実行後に DB の不変条件を検証する。違反があれば
-- RAISE_APPLICATION_ERROR で失敗させ、runnora を exit 4 で終了させる。
--

```

```
-- 不変条件:
--   (1) 蔵書ごとに 貸出可能冊数 = 所蔵冊数 - 貸出中件数
--   (2) ACTIVE の貸出は返却日なし / RETURNED の貸出は返却日あり
--   (3) 会員の貸出中件数 <= 貸出上限
--
-- 使用構文: カーソル FOR LOOP / CASE 文 / 局所プロシージャ
--           PRAGMA AUTONOMOUS_TRANSACTION / RAISE_APPLICATION_ERROR
-- =====
DECLARE
  v_errors PLS_INTEGER := 0;
  v_checked PLS_INTEGER := 0;
  v_detail VARCHAR2(4000);

  PROCEDURE add_error(p_message IN VARCHAR2) IS
  BEGIN
    v_errors := v_errors + 1;
    IF NVL(LENGTH(v_detail), 0) < 3000 THEN
      v_detail := v_detail || p_message || ',';
    END IF;
  END add_error;

  PROCEDURE log_hook(p_message IN VARCHAR2) IS
  PRAGMA AUTONOMOUS_TRANSACTION;
  BEGIN
    INSERT INTO test_hook_log (hook_name, message) VALUES ('common/90_verify_integrity', p_message);
    COMMIT;
  END log_hook;
BEGIN
  -- (1) 在庫数の整合 : カーソル FOR LOOP
  FOR r IN (
    SELECT b.book_id, b.total_copies, b.available_copies, COUNT(l.loan_id) AS active_loans
      FROM books b
     LEFT JOIN loans l ON l.book_id = b.book_id AND l.status = 'ACTIVE'
    GROUP BY b.book_id, b.total_copies, b.available_copies
    ORDER BY b.book_id
  ) LOOP
    v_checked := v_checked + 1;
    IF r.available_copies <> r.total_copies - r.active_loans THEN
      add_error(r.book_id || ' available=' || r.available_copies
                || ' expected=' || (r.total_copies - r.active_loans));
    END IF;
  END LOOP;
```

```
-- (2) 貸出状態と返却日の整合 : CASE 文
FOR r IN (SELECT loan_id, status, returned_at FROM loans ORDER BY loan_id) LOOP
    CASE r.status
        WHEN 'ACTIVE' THEN
            IF r.returned_at IS NOT NULL THEN
                add_error('loan ' || r.loan_id || ' is ACTIVE but has returned_at');
            END IF;
        WHEN 'RETURNED' THEN
            IF r.returned_at IS NULL THEN
                add_error('loan ' || r.loan_id || ' is RETURNED but returned_at is null');
            END IF;
        END CASE;
    END LOOP;

-- (3) 貸出上限
FOR r IN (
    SELECT m.member_id, m.max_loans, COUNT(l.loan_id) AS active_loans
    FROM members m
    JOIN loans l ON l.member_id = m.member_id AND l.status = 'ACTIVE'
    GROUP BY m.member_id, m.max_loans
    HAVING COUNT(l.loan_id) > m.max_loans
) LOOP
    add_error(r.member_id || ' has ' || r.active_loans || ' loans (max ' || r.max_loans || ')');
END LOOP;

log_hook('books checked=' || v_checked || ', errors=' || v_errors);

IF v_errors > 0 THEN
    RAISE_APPLICATION_ERROR(-20100, 'integrity check failed (' || v_errors || '): ' || v_detail);
END IF;

END;
```

8.2 sql/cases/

8.2.1 contract_setup.sql

```
-- =====
-- cases/contract_setup.sql (--before-sql : 契約テスト・生成テスト用)
-- モック (oapi2wire) と同じ前提データを実 DB に作る。
--   loan 1 : M0002 が B0003 を貸出中 (ACTIVE)   → POST /loans/1/return は 200
--   loan 2 : M0002 が B0004 を返却済 (RETURNED) → POST /loans/2/return は 409
--
-- 使用構文: 行ロック (SELECT ... FOR UPDATE) / RETURNING INTO / 局所プロシージャ
-- =====

DECLARE
  v_loan_id loans.loan_id%TYPE;
  v_avail   books.available_copies%TYPE;

  PROCEDURE lend(p_member IN VARCHAR2, p_book IN VARCHAR2, p_days_ago IN PLS_INTEGER,
                 p_returned IN BOOLEAN, p_loan_id OUT NUMBER) IS

  BEGIN
    -- API と同じく蔵書行をロックしてから在庫を動かす
    SELECT available_copies INTO v_avail FROM books WHERE book_id = p_book FOR UPDATE;

    INSERT INTO loans (loan_id, member_id, book_id, loaned_at, due_date, returned_at, status)
    VALUES (loan_seq.NEXTVAL, p_member, p_book,
            TRUNC(SYSDATE) - p_days_ago, TRUNC(SYSDATE) - p_days_ago + 14,
            CASE WHEN p_returned THEN TRUNC(SYSDATE) - p_days_ago + 9 END,
            CASE WHEN p_returned THEN 'RETURNED' ELSE 'ACTIVE' END)
    RETURNING loan_id INTO p_loan_id;

    IF NOT p_returned THEN
      UPDATE books SET available_copies = v_avail - 1 WHERE book_id = p_book;
    END IF;
  END lend;

BEGIN
  lend('M0002', 'B0003', 6, FALSE, v_loan_id); -- loan 1 (ACTIVE)
  lend('M0002', 'B0004', 25, TRUE, v_loan_id);  -- loan 2 (RETURNED)
  COMMIT;
END;
```

8.2.2 demo_break_invariant_after.sql

```
-- =====
-- cases/demo_break_invariant_after.sql (--after-sql : 検知デモ専用)
-- わざと在庫数を 1 減らして不変条件を壊す。
-- --after-sql は common.after より先に実行されるので、
-- 直後の common/90_verify_integrity.sql が不整合を検知して exit 4 になることを確認できる。
```

```
-- 通常のテストでは使わない (scripts/test-api.ps1 の「検知確認」でのみ使用)。
```

```
-- =====
```

```
BEGIN
```

```
    UPDATE books SET available_copies = available_copies - 1 WHERE book_id = 'B0001';
```

```
    COMMIT;
```

```
END;
```

8.2.3 lib001_assert_after.sql

```
-- =====
```

```
-- cases/lib001_assert_after.sql (--after-sql : LIB-001 貸出～返却)
```

```
-- API 経由では見えない DB 上の事後条件を検証する。
```

```
--   - 貸出は 1 件だけ作られ、RETURNED・返却日 = 本日 になっている
```

```
--   - B0001 の貸出可能冊数が初期値 3 に戻っている
```

```
--
```

```
-- 使用構文: SELECT INTO / 例外 NO_DATA_FOUND・TOO_MANY_ROWS
```

```
--           ユーザー定義例外 (EXCEPTION + RAISE) / RAISE_APPLICATION_ERROR
```

```
-- =====
```

```
DECLARE
```

```
    e_assert    EXCEPTION;
```

```
    v_message   VARCHAR2(400);
```

```
    v_loan      loans%ROWTYPE;
```

```
    v_avail     books.available_copies%TYPE;
```

```
BEGIN
```

```
-- 1 件だけであることを SELECT INTO の例外で確かめる
```

```
SELECT * INTO v_loan FROM loans WHERE member_id = 'M0001';
```

```
IF v_loan.loan_id <> 1 OR v_loan.book_id <> 'B0001' THEN
```

```
    v_message := 'unexpected loan: id=' || v_loan.loan_id || ' book=' || v_loan.book_id;
```

```
    RAISE e_assert;
```

```
ELSIF v_loan.status <> 'RETURNED' OR v_loan.returned_at <> TRUNC(SYSDATE) THEN
```

```
    v_message := 'loan 1 is not returned today: status=' || v_loan.status
```

```
                || ' returned_at=' || TO_CHAR(v_loan.returned_at, 'YYYY-MM-DD');
```

```
    RAISE e_assert;
```

```
END IF;
```

```
SELECT available_copies INTO v_avail FROM books WHERE book_id = 'B0001';
```

```
IF v_avail <> 3 THEN
```

```
    v_message := 'B0001 available_copies=' || v_avail || ' (expected 3)';
```

```
    RAISE e_assert;
```

```
END IF;
```

```
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN
```

```
        RAISE_APPLICATION_ERROR(-20001, 'LIB-001: no loan found for M0001');
```

```
WHEN TOO_MANY_ROWS THEN
    RAISE_APPLICATION_ERROR(-20001, 'LIB-001: more than one loan found for M0001');
WHEN e_assert THEN
    RAISE_APPLICATION_ERROR(-20001, 'LIB-001: ' || v_message);
END;
```

8.2.4 lib003_overdue_before.sql

```
-- =====
-- cases/lib003_overdue_before.sql  (--before-sql : LIB-003 延滞者の貸出拒否)
-- API では作れない「返却期限を 7 日過ぎた貸出」を直接作る。
--   loan 1 : M0002 が B0004 を 21 日前に借りて未返却（期限は 7 日前）
--
-- 使用構文: %ROWTYPE レコード / レコード単位の INSERT (VALUES rec)
--           PL/SQL 式でのシーケンス参照 / 日付演算 / RETURNING INTO
-- =====

DECLARE
    v_loan  loans%ROWTYPE;
    v_title books.title%TYPE;
BEGIN
    v_loan.loan_id      := loan_seq.NEXTVAL;
    v_loan.member_id    := 'M0002';
    v_loan.book_id       := 'B0004';
    v_loan.loaned_at     := TRUNC(SYSDATE) - 21;
    v_loan.due_date      := v_loan.loaned_at + 14;    -- = 7 日前
    v_loan.returned_at   := NULL;
    v_loan.status        := 'ACTIVE';

    INSERT INTO loans VALUES v_loan;

    -- 不変条件（貸出可能冊数 = 所蔵 - 貸出中）を保つために在庫も減らす
    UPDATE books
        SET available_copies = available_copies - 1
        WHERE book_id = v_loan.book_id
    RETURNING title INTO v_title;

    IF SQL%ROWCOUNT <> 1 THEN
        RAISE_APPLICATION_ERROR(-20003, 'LIB-003: book ' || v_loan.book_id || ' not found');
    END IF;
    COMMIT;
END;
```

8.2.5 lib004_fill_loans_before.sql

```
-- =====
-- cases/lib004_fill_loans_before.sql  (--before-sql : LIB-004 貸出上限)
-- M0001 に上限 (max_loans = 3) まで TECH の本を貸し出した状態を作る。
-- 貸出可能な本を book_id 順に探すので loan 1..3 = B0003, B0006, B0008 になる。
--
-- 使用構文: パラメータ付き明示カーソル (OPEN / FETCH / CLOSE)
--          %NOTFOUND / %ISOPEN / WHILE LOOP / EXIT WHEN
-- =====

DECLARE
  CURSOR c_available (p_genre VARCHAR2) IS
    SELECT book_id
      FROM books
     WHERE genre = p_genre
        AND available_copies > 0
     ORDER BY book_id
    FOR UPDATE;

  v_member  CONSTANT members.member_id%TYPE := 'M0001';
  v_max     members.max_loans%TYPE;
  v_count   PLS_INTEGER := 0;
  v_book_id books.book_id%TYPE;
BEGIN
  SELECT max_loans INTO v_max FROM members WHERE member_id = v_member;

  OPEN c_available('TECH');
  WHILE v_count < v_max LOOP
    FETCH c_available INTO v_book_id;
    EXIT WHEN c_available%NOTFOUND;  -- 貸出可能な本が尽きたら終了

    INSERT INTO loans (loan_id, member_id, book_id, loaned_at, due_date, status)
      VALUES (loan_seq.NEXTVAL, v_member, v_book_id, TRUNC(SYSDATE) - 1, TRUNC(SYSDATE) + 13, 'ACTIVE');

    UPDATE books SET available_copies = available_copies - 1
      WHERE CURRENT OF c_available;

    v_count := v_count + 1;
  END LOOP;
  IF c_available%ISOPEN THEN
    CLOSE c_available;
  END IF;

  IF v_count < v_max THEN
```

```
RAISE_APPLICATION_ERROR(-20004, 'LIB-004: only ' || v_count || ' books could be lent');  
END IF;  
COMMIT;  
END;
```

8.2.6 lib005_assert_after.sql

```
-- =====  
-- cases/lib005_assert_after.sql (--after-sql : LIB-005 SAVEPOINT)  
-- SAVEPOINT で取り消した単位が残っていないこと、API で再登録した B0103 が  
-- 追加されていることを DB で確認する。  
--  
-- 使用構文: 集計の SELECT INTO / 条件付き COUNT / RAISE_APPLICATION_ERROR  
-- =====  
  
DECLARE  
    v_test_books PLS_INTEGER;  
    v_b0104      PLS_INTEGER;  
    v_rollbacks  PLS_INTEGER;  
  
BEGIN  
    SELECT COUNT(*), COUNT(CASE WHEN book_id = 'B0104' THEN 1 END)  
        INTO v_test_books, v_b0104  
        FROM books  
        WHERE genre = 'TEST';  
  
    -- 前処理で 3 件 (B0101, B0102, B0105) + シナリオ内の API 登録 1 件 (B0103)  
    IF v_test_books <> 4 THEN  
        RAISE_APPLICATION_ERROR(-20005, 'LIB-005: TEST books=' || v_test_books || ' (expected 4)');  
    END IF;  
  
    IF v_b0104 <> 0 THEN  
        RAISE_APPLICATION_ERROR(-20005, 'LIB-005: B0104 should have been rolled back');  
    END IF;  
  
    -- 自律型トランザクションのログは ROLLBACK TO の影響を受けずに残っている  
    SELECT COUNT(*) INTO v_rollbacks  
        FROM test_hook_log  
        WHERE hook_name = 'cases/lib005_savepoint'  
            AND message LIKE 'rollback B%'  
            AND logged_at > SYSTIMESTAMP - INTERVAL '10' MINUTE;  
  
    IF v_rollbacks < 2 THEN  
        RAISE_APPLICATION_ERROR(-20005, 'LIB-005: rollback log count=' || v_rollbacks || ' (expected >= 2)');  
    END IF;  
END;
```

8.2.7 lib005_savepoint_before.sql

```
-- =====
-- cases/lib005_savepoint_before.sql  (--before-sql : LIB-005 SAVEPOINT)
-- 「蔵書登録 + 初回貸出」を 1 単位として 5 件投入し、失敗した単位だけを
-- ROLLBACK TO SAVEPOINT で取り消して、成功した単位は最後にまとめて COMMIT する。
--
-- 単位 | 蔵書 | ISBN | 冊数 | 初回貸出 | 結果
-- -----+-----+-----+-----+-----+-----
-- 1 | B0101 | 9784000000101 | 2 | M0004 | 登録 (loan 1)
-- 2 | B0102 | 9784000000102 | 1 | - | 登録
-- 3 | B0103 | 9784000000101 | 1 | - | ISBN 重複 → 取消
-- 4 | B0104 | 9784000000104 | 1 | M9999 | 蔵書は入るが貸出で FK 違反 → 蔵書ごと取消
--                                     (loan 2 の採番は消費されたまま)
-- 5 | B0105 | 9784000000105 | 3 | M0005 | 登録 (loan 3)
--
-- 後半は「一括の利用停止が想定件数を超えたら取り消し、個別停止に切り替える」例。
--
-- 使用構文: SAVEPOINT / ROLLBACK TO SAVEPOINT / ネストしたブロックの例外処理
--           DUP_VAL_ON_INDEX / PRAGMA EXCEPTION_INIT / 修飾式 / SQL%ROWCOUNT
--           PRAGMA AUTONOMOUS_TRANSACTION (ROLLBACK されないログ)
-- =====

DECLARE
    TYPE t_unit IS RECORD (
        book_id books.book_id%TYPE,
        isbn     books.isbn%TYPE,
        copies   books.total_copies%TYPE,
        lend_to  members.member_id%TYPE
    );
    TYPE t_units IS TABLE OF t_unit INDEX BY PLS_INTEGER;

    e_parent_missing EXCEPTION;
    PRAGMA EXCEPTION_INIT(e_parent_missing, -2291); -- ORA-02291: 親キーがありません

    v_units      t_units;
    v_loan_id     loans.loan_id%TYPE;
    v_ok          PLS_INTEGER := 0;
    v_rolled      PLS_INTEGER := 0;
    v_suspended   PLS_INTEGER;

    PROCEDURE log_hook(p_message IN VARCHAR2) IS
        PRAGMA AUTONOMOUS_TRANSACTION;
    BEGIN
        INSERT INTO test_hook_log (hook_name, message) VALUES ('cases/lib005_savepoint', p_message);
```

```
COMMIT;
END log_hook;
BEGIN
  v_units(1) := t_unit('B0101', '9784000000101', 2, 'M0004');
  v_units(2) := t_unit('B0102', '9784000000102', 1, NULL);
  v_units(3) := t_unit('B0103', '9784000000101', 1, NULL);      -- ISBN 重複
  v_units(4) := t_unit('B0104', '9784000000104', 1, 'M9999');  -- 存在しない会員
  v_units(5) := t_unit('B0105', '9784000000105', 3, 'M0005');

  FOR i IN 1 .. v_units.COUNT LOOP
    SAVEPOINT sp_unit;    -- 同名 SAVEPOINT は再設定すると位置が上書きされる
    BEGIN
      INSERT INTO books (book_id, isbn, title, author, genre, total_copies, available_copies)
      VALUES (v_units(i).book_id, v_units(i).isbn, 'SAVEPOINT 検証' || v_units(i).book_id,
              '試験 花子', 'TEST', v_units(i).copies, v_units(i).copies);

      IF v_units(i).lend_to IS NOT NULL THEN
        INSERT INTO loans (loan_id, member_id, book_id, loaned_at, due_date, status)
        VALUES (loan_seq.NEXTVAL, v_units(i).lend_to, v_units(i).book_id,
                TRUNC(SYSDATE), TRUNC(SYSDATE) + 14, 'ACTIVE')
        RETURNING loan_id INTO v_loan_id;

        UPDATE books SET available_copies = available_copies - 1
        WHERE book_id = v_units(i).book_id;
      END IF;
      v_ok := v_ok + 1;
    EXCEPTION
      WHEN DUP_VAL_ON_INDEX THEN
        ROLLBACK TO SAVEPOINT sp_unit;
        v_rolled := v_rolled + 1;
        log_hook('rollback ' || v_units(i).book_id || ': duplicate isbn ' || v_units(i).isbn);
      WHEN e_parent_missing THEN
        -- 蔵書 INSERT は成功済み。SAVEPOINT まで戻すことで蔵書も一緒に取り消される
        ROLLBACK TO SAVEPOINT sp_unit;
        v_rolled := v_rolled + 1;
        log_hook('rollback ' || v_units(i).book_id || ': member ' || v_units(i).lend_to || ' not found');
      END;
    END LOOP;

    -- 後半：一括停止 → 件数超過なら取り消して個別停止
    SAVEPOINT sp_before_suspend;
    UPDATE members SET status = 'SUSPENDED' WHERE member_id IN ('M0006', 'M0007', 'M0008');
    v_suspended := SQL%ROWCOUNT;    -- ROLLBACK 後は参照できないので先に退避する
```

```

IF v_suspended > 1 THEN
    ROLLBACK TO SAVEPOINT sp_before_suspend;
    log_hook('rollback bulk suspend (' || v_suspended || ' rows); suspend M0008 only');
    UPDATE members SET status = 'SUSPENDED' WHERE member_id = 'M0008';
END IF;

COMMIT;
log_hook('units ok=' || v_ok || ', rolled back=' || v_rolled);
END;

```

8.2.8 lib006_assert_no_loans_after.sql

```

-- =====
-- cases/lib006_assert_no_loans_after.sql (--after-sql : LIB-006 異常系)
-- エラー応答を返したリクエストが DB を変更していないことを確認する。
--   - 貸出は 0 件 / 蔵書は初期の 20 件 / 採番は一度も進んでいない
--
-- 使用構文: SELECT INTO / USER_SEQUENCES の参照 / IF ... ELSIF
-- =====
DECLARE
    v_loans  PLS_INTEGER;
    v_books  PLS_INTEGER;
    v_nextno NUMBER;
BEGIN
    SELECT COUNT(*) INTO v_loans FROM loans;
    SELECT COUNT(*) INTO v_books FROM books;
    -- NOCACHE なので LAST_NUMBER = 次に払い出される値
    SELECT last_number INTO v_nextno FROM user_sequences WHERE sequence_name = 'LOAN_SEQ';

    IF v_loans <> 0 THEN
        RAISE_APPLICATION_ERROR(-20006, 'LIB-006: loans=' || v_loans || ' (expected 0)');
    ELSIF v_books <> 20 THEN
        RAISE_APPLICATION_ERROR(-20006, 'LIB-006: books=' || v_books || ' (expected 20)');
    ELSIF v_nextno <> 1 THEN
        RAISE_APPLICATION_ERROR(-20006, 'LIB-006: loan_seq advanced to ' || v_nextno);
    END IF;
END;

```

8.2.9 lib007_history_before.sql

```

-- =====
-- cases/lib007_history_before.sql (--before-sql : LIB-007 大量の貸出履歴)
-- M0004 に、B0005～B0020 の 16 冊分の返却済み貸出履歴を一括投入する。
-- 5 件ずつ BULK COLLECT で読み、FORALL でまとめて INSERT する「バッチ処理」の定型。
--   loan 1..16 : 全て RETURNED (loaned_at は 1 週間ずつずらす)

```

```
--
-- 使用構文: BULK COLLECT ... LIMIT / FORALL / SQL%BULK_ROWCOUNT
--          入れ子表 (TABLE OF) / 基本 LOOP + EXIT WHEN
-- =====
DECLARE
    CURSOR c_books IS
        SELECT book_id
          FROM books
         WHERE book_id BETWEEN 'B0005' AND 'B0020'
         ORDER BY book_id;

    TYPE t_ids   IS TABLE OF books.book_id%TYPE;
    TYPE t_dates IS TABLE OF DATE;
    v_ids       t_ids;
    v_loaned    t_dates := t_dates();
    v_offset    PLS_INTEGER := 0;
    v_batches   PLS_INTEGER := 0;
    v_rows      PLS_INTEGER := 0;

    PROCEDURE log_hook(p_message IN VARCHAR2) IS
        PRAGMA AUTONOMOUS_TRANSACTION;
    BEGIN
        INSERT INTO test_hook_log (hook_name, message) VALUES ('cases/lib007_history', p_message);
        COMMIT;
    END log_hook;
BEGIN
    OPEN c_books;
    LOOP
        FETCH c_books BULK COLLECT INTO v_ids LIMIT 5;
        EXIT WHEN v_ids.COUNT = 0;
        v_batches := v_batches + 1;

        -- 貸出日を事前に計算 (FORALL の中では式に添字しか使えないため)
        v_loaned.DELETE;
        v_loaned.EXTEND(v_ids.COUNT);
        FOR i IN 1 .. v_ids.COUNT LOOP
            v_loaned(i) := TRUNC(SYSDATE) - 200 + (v_offset + i) * 7;
        END LOOP;

        FORALL i IN 1 .. v_ids.COUNT
            INSERT INTO loans (loan_id, member_id, book_id, loaned_at, due_date, returned_at, status)
              VALUES (loan_seq.NEXTVAL, 'M0004', v_ids(i), v_loaned(i), v_loaned(i) + 14, v_loaned(i) + 10,
                'RETURNED');
```

```
FOR i IN 1 .. v_ids.COUNT LOOP
    v_rows := v_rows + SQL%BULK_ROWCOUNT(i);
END LOOP;
v_offset := v_offset + v_ids.COUNT;
END LOOP;
CLOSE c_books;

IF v_rows <> 16 THEN
    RAISE_APPLICATION_ERROR(-20007, 'LIB-007: inserted ' || v_rows || ' rows (expected 16)');
END IF;
COMMIT;
log_hook('batches=' || v_batches || ', rows=' || v_rows);
END;
```